

VILNIAUS UNIVERSITETAS

MATEMATIKOS IR INFORMATIKOS FAKULTETAS

DUOMENŲ MOKSLO STUDIJŲ PROGRAMA

1-as laboratorinis darbas

Binarinio atsako regresijos modeliai

Naglis Bražėnas, Arnas Kazanavičius, Justina Seiliūnaitė, Arnas Usonis

Doc., Dr. Rūta Levulienė

Vilnius
2026

Turinys

Iliustracijų sąrašas **2**

Lentelių sąrašas **3**

Įvadas **1**

1. Programinė aplinka ir naudoti įrankiai **2**

2. Duomenų aprašymas ir apdorojimas **3**

2.1. Kintamųjų aprašymas 3

2.2. Duomenų apdorojimas 3

3. Pradinė kintamųjų analizė **5**

4. Modeliai **14**

4.1. *logit* modelis 14

4.2. *probit* modelis 17

4.3. *cloglog* modelis 19

5. Modelio tobulinimas **21**

5.1. *Probit* modelis su vidutinio gliukozės lygio ir širdies ligų sąveika 22

6. Geriausio modelio interpretacija **25**

6.1. 95 % pasikliovimo intervalai (*probit* modelis) 25

6.2. Marginalieji efektai (*probit* modelis) 26

Išvados **27**

Rekomendacijos **28**

Literatūros sąrašas **29**

Priedas A: Pilnas R kodas **30**

Priedas B: Pilnas Python kodas **38**

Iliustracijų sąrašas

1 KMI pasiskirstymas pagal insultą 5
2 Amžiaus pasiskirstymas pagal insultą 6
3 Vidutinio gliukozes kiekio pasiskirstymas pagal insultą 7
4 KMI pasiskirstymas 8
5 Amžiaus pasiskirstymas 8
6 Vidutinio gliukozės kiekio pasiskirstymas 9
7 KMI tankis pagal insultą 10
8 Amžiaus tankio grafikas pagal insultą 11
9 Vidutinio gliukozės kiekio tankis pagal insultą 12
10 „Cook“ mato grafikas 15
11 „Pearson“ liekanų grafikas 15
12 ROC kreivė *logit* modeliui testavimo aibei 17
13 ROC kreivė *probit* modeliui testavimo aibei 19
14 ROC kreivė *cloglog* modeliui testavimo aibei 20
15 ROC kreivė pažingsniniam *probit* modeliui testavimo aibei 22
16 ROC kreivė testavimo aibei (probit) modeliui su sąveika 23

Lentelių sąrašas

1 Insulto dažnių lentelė 5
2 Kiekybinių kintamųjų min ir max 5
3 Kategorinių kintamųjų dažniai duomenų rinkinyje 12
4 Kategorinių kintamųjų pasiskirstymas pagal insulto reikšmę 13
5 Priklausomo kintamojo (insulto) pasiskirstymas mokymo aibėje 14
6 Priklausomo kintamojo (insulto) pasiskirstymas testavimo aibėje 14
7 Multikolinearumas 16
8 Klasifikavimo lentelė *logit* modeliui testavimo aibei 16
9 Klasifikavimo tikslumas *logit* modeliui 17
10 Klasifikavimo lentelė *probit* modeliui testavimo aibei 18
11 Klasifikavimo tikslumas 18
12 Klasifikavimo lentelė *cloglog* testavimo aibei 19
13 Klasifikavimo tikslumas 20
14 Klasifikavimo lentelė *probit* modeliui testavimo aibei 21
15 Klasifikavimo tikslumas 21
16 Klasifikavimo lentelė testavimo aibei (probit su sąveika) 23
17 Klasifikavimo tikslumas 23
18 95 % pasikliovimo intervalai (*probit* modelis) 25
19 Vidutiniai marginalieji efektai (*probit* modelis) 26

Įvadas

Šiuolaikinėje medicinos duomenų analizėje insulto prognozavimas tampa vis svarbesnis, nes laiku atpažįstami rizikos veiksniai leidžia užkirsti kelią sunkiems sveikatos sutrikimams. Insulto atsiradimui įtaką gali daryti amžius, lytis, kūno masės indeksas (KMI), vidutinis gliukozės kiekis kraujyje, hipertenzijos ir širdies ligų buvimas bei gyvenimo būdo ypatybės. Todėl galima tikėtis, kad skirtingi paciento gyvenimo būdo ir sveikatos veiksniai gali daryti įtaką insulto rizikai ir suteikti galimybę prognozuoti, kurie pacientai yra didesnės rizikos grupėje.

Tyrimo tikslas

Šio darbo tikslas yra pritaikyti ir palyginti binarinio atsako regresijos modelius insulto tikimybei prognozuoti, naudojant demografinius, gyvenimo būdo ir sveikatos rodiklius, bei nustatyti tiksliausią modelį.

Tyrimo uždaviniai

1. Paruošti ir išvalyti insulto prognozavimo duomenų rinkinį binariniam klasifikavimui;
2. Atlikti pradinę insulto prognozavimo duomenų analizę, įvertinti kintamųjų pasiskirstymą ir ryšį su priklausomuoju kintamuoju;
3. Identifikuoti, ar insulto prognozavimo duomenyse yra išskirčių ir įvertinti multikolinearumą tarp kovariančių;
4. Sudaryti binarinio atsako regresijos modelius (*logit*, *probit* ir *cloglog*) insulto tikimybei prognozuoti;
5. Įvertinti sudarytų modelių insulto klasifikavimo kokybę;
6. Palyginti modelius ir nustatyti tiksliausiai insulto riziką prognozuojantį modelį;
7. Apskaičiuoti ir interpretuoti galutinio modelio vidutinius marginaliuosius efektus ir įvertinti svarbiausių veiksnių poveikį insulto tikimybei.

1. Programinė aplinka ir naudoti įrankiai

Analizė atlikta naudojant atvirojo kodo statistinės analizės programinę aplinką **R**.

Naudotos bibliotekos

Analizei naudoti pagrindiniai R paketai ir jų paskirtys:

- `dplyr` – duomenų apdorojimas ir tvarkymas;
- `ggplot2` – duomenų vizualizavimas;
- `pROC` – ROC kreivės ir klasifikavimo modelių tikslumo vertinimas;
- `car` – regresijos modelių diagnostika;
- `caret` - biblioteka naudojama mašininio mokymosi modelių kūrimui ir vertinimui.

LaTeX dokumento generavimas

Rašto darbo dokumentas sukurtas naudojant internetinę LaTeX redagavimo aplinką **Overleaf**. Ši platforma leido bendradarbiauti, kontroliuoti versijas ir patogiai integruoti R-grafikus bei lenteles į galutinį PDF formatą.

2. Duomenų aprašymas ir apdorojimas

Duomenų rinkinys *„Stroke Prediction Dataset"* gautas iš viešai prieinamo šaltinio *„Kaggle"*, kuris talpina įvairius vartotojų pateiktus atvirus duomenis analizei. Šiame duomenų rinkinyje pateikiami pacientų medicininiai bei gyvenimo būdo duomenys, kurie padeda analizuoti veiksnius, galinčius turėti įtakos insulto atsiradimui, bei kurti modelius insulto rizikai prognozuoti. Pradiniame duomenų rinkinyje yra 5110 stebėjimų.

2.1. Kintamųjų aprašymas

Lytis *(angl. gender)* – kategorinis kintamasis, nusakantis tiriamojo asmens lytį.

Amžius *(angl. age)* – kiekybinis kintamasis, nusakantis tiriamojo amžių metais.

Hipertenzija *(angl. hypertension)* – kategorinis kintamasis, nusakantis ar asmuo serga hipertenzija (turi aukštą kraujospūdį). Kintamasis įgyja dvi reikšmes: 0 – hipertenzijos nėra, 1 – hipertenzija nustatyta.

Širdies ligos *(angl. heart disease)* – kategorinis kintamasis, nusakantis ar asmuo serga širdies ligomis. Kintamasis įgyja dvi reikšmes: 0 - širdies ligų nėra, 1 – širdies liga nustatyta.

Santuoka *(angl. ever married)* – kategorinis kintamasis, nusakantis ar asmuo yra kada nors buvęs susituokęs. Kintamasis turi dvi reikšmes: „Yes" – buvo susituokęs, „No" – nebuvo susituokęs.

Darbo tipas *(angl. work type)* – kategorinis kintamasis, nusakantis asmens darbo tipą. Kintamasis apibūdina darbo kategoriją, pavyzdžiui: privatus sektorius, individuali veikla, valstybės tarnyba ir kt.

Gyvenvietės tipas *(angl. residence type)* – kategorinis kintamasis, nusakantis gyvenamosios vietos tipą. Kintamasis turi dvi reikšmes: „Urban" – miestas, „Rural" – kaimo vietovė.

Vidutinis gliukozės kiekis *(angl. average glucose level)* – kiekybinis kintamasis, nusakantis vidutinį gliukozės kiekį kraujyje.

KMI *(angl. BMI)* – kiekybinis kintamasis, nusakantis kūno masės indeksą.

Rūkymo statusas *(angl. smoking status)* – kategorinis kintamasis, nusakantis asmens rūkymo statusą (nerūkantis, metęs rūkyti, rūkantis).

Insultas *(angl. stroke)* - kategorinis kintamasis, nusakantis ar asmuo patyrė insultą. Jį pasirinkome kaip priklausomą kintamąjį. Kintamasis įgyja dvi reikšmes: 0 – insulto nebuvo, 1 – insultas nustatytas.

2.2. Duomenų apdorojimas

Įsikėlus duomenis pirmiausia, kadangi ID kintamasis naudojamas tik stebėjimų identifikavimui ir neturi analitinės reikšmės, jis pašalinamas iš duomenų rinkinio. Vėliau atliekamas duomenų valymas. Iš duomenų rinkinio pašalinamos visos eilutės, kuriose yra trūkstamų reikšmių. Taip pat pašalinami įrašai, kuriuose KMI *(angl. BMI)* reikšmė pateikta kaip tekstinė reikšmė „N/A". Be to, iš duomenų rinkinio pašalinami įrašai, kuriuose rūkymo statusas yra nežinomas, kadangi tokių reikšmių buvo apie

30 % visų duomenų rinkinio įrašų. Papildomai pašalinami įrašai su lytimi „Other“, kurių buvo apie 1 %. Atlikti analizę apsibrėžėme nenagrinėti reikšmių, kurių darbo tipo kintamasis buvo „children“ arba „never worked“, kadangi šios reikšmės sudarė apie 2 % visų duomenų ir neturėjo insulto reikšmės.

3. Pradinė kintamųjų analizė

Siekiant geriau suprasti turimus duomenis, atlikta pradinė kintamųjų analizė. Tai svarbu, nes nesubalansuotas duomenų rinkinys gali iškreipti binarinio atsako regresijos modelio rezultatus, ypač mažesnės klasės prognozavimo tikslumą.

1 lentelė. *Insulto dažnių lentelė*

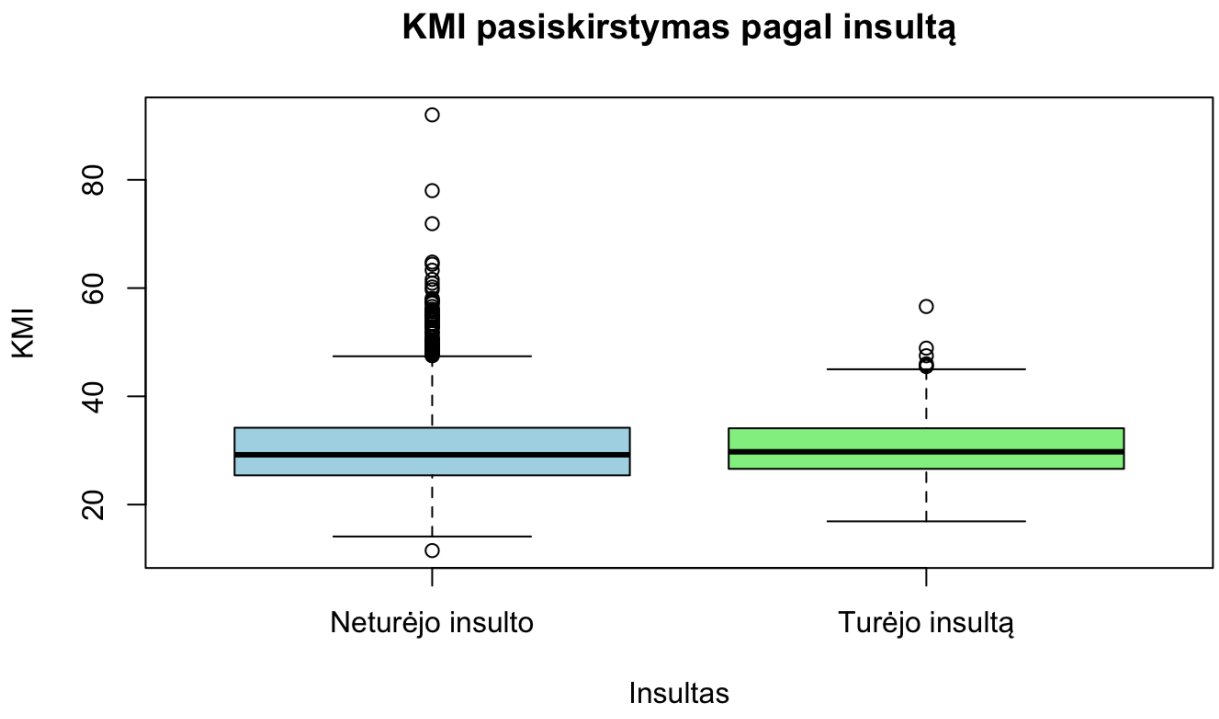
Turėjo insultą	Neturėjo insulto
180	3163

Iš lentelės *(žr. 1 lentelėje)* matome, kad insultą turėjusių pacientų skaičius yra nesubalansuotas, todėl susiduriame su nesubalansuotų klasių problema.

2 lentelė. *Kiekybinių kintamųjų min ir max*

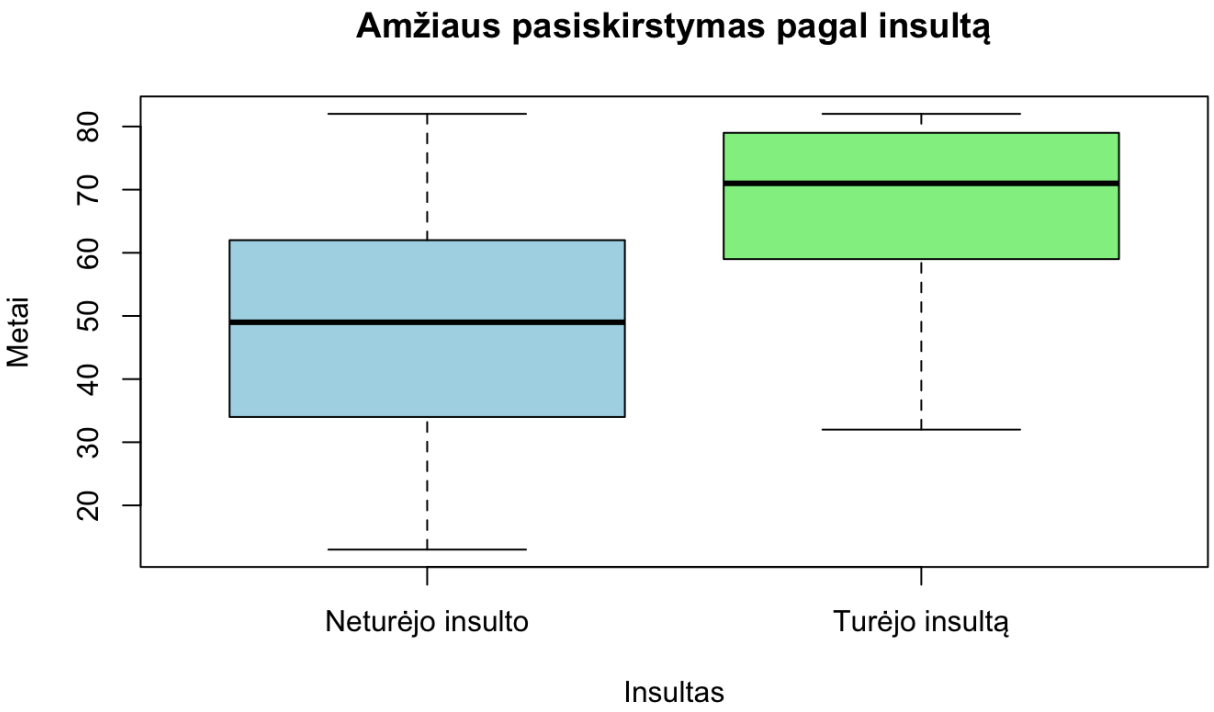
Kintamasis	Min	Max
Amžius	13	82
KMI	11,5	92
Vidutinis gliukozės kiekis	55,12	271,74

Galima matyti *(žr. 2 lentelėje)*, kad pacientų amžiaus intervalas yra gana platus (13–82 metai). KMI svyruoja nuo 11,5 iki 92, o vidutinis gliukozės kiekis nuo 55,12 iki 271,74. Tiek KMI tiek vidutinio gliukozės kiekio intervalai taip pat yra gana platūs.



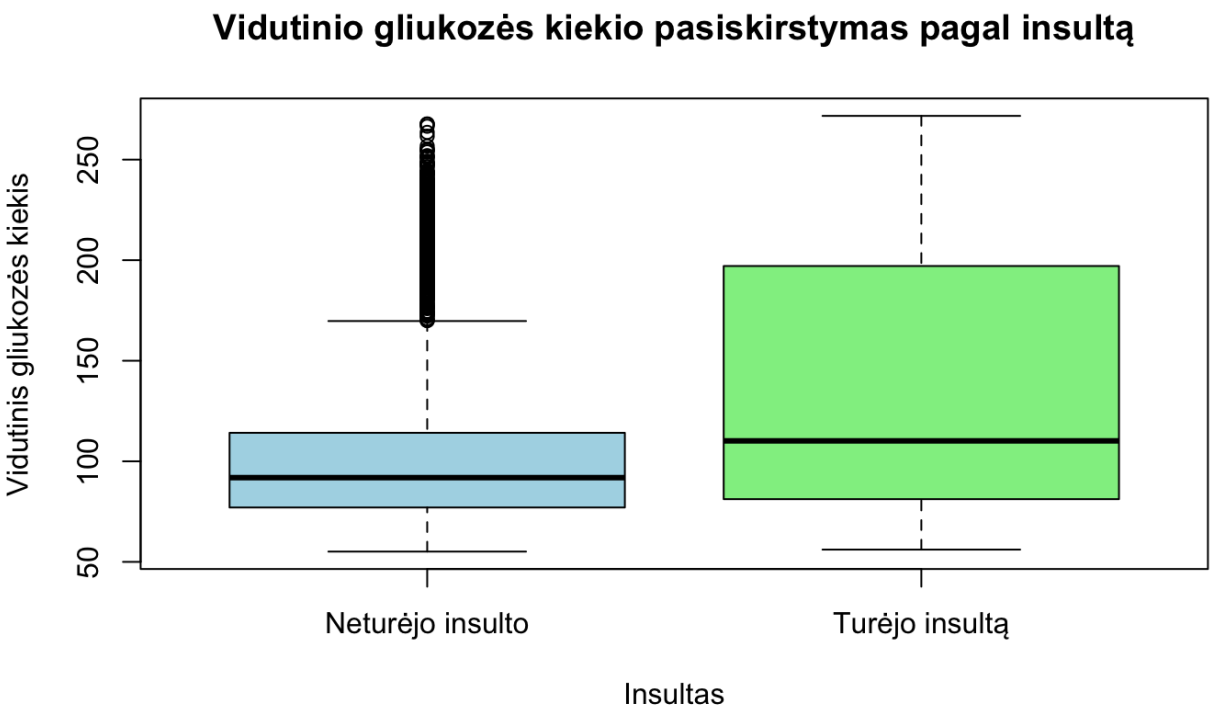
1 pav. *KMI pasiskirstymas pagal insultą*

Matoma *(žr. 1 pav.)*, kad tiek patyrusių insultą, tiek nepatyrusių pacientų KMI yra gan panašiai pasiskirstęs. Taip pat galima pastebėti, kad tarp insulto nepatyrusių pacientų KMI yra nemažai išskirtinių reikšmių. Panašus pasiskirstymas gali rodyti, kad žmogaus kūno masės indeksas gali nedaryti įtakos insultui.



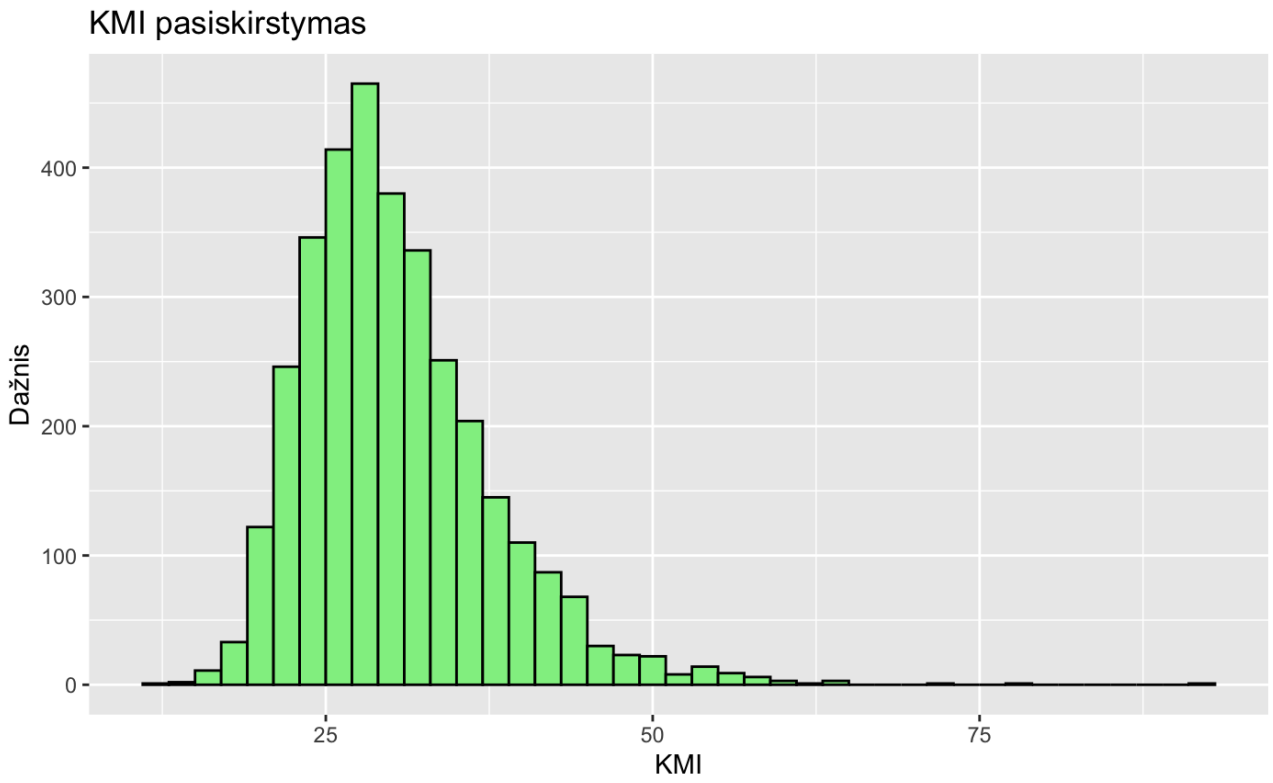
2 pav. *Amžiaus pasiskirstymas pagal insultą*

Iš grafiko *(žr. 2 pav.)* matyti, kad išskirtinių taškų nėra ir neturėjusių insulto amžius pasiskirstęs gan plačiai. Taip pat matome, kad turėjusių insultą amžius yra didesnis už neturėjusių, o tai rodo, kad insultą turėję pacientai yra vyresni. Tai gali reikšti, kad amžius gali daryti įtaką insulto rizikai.



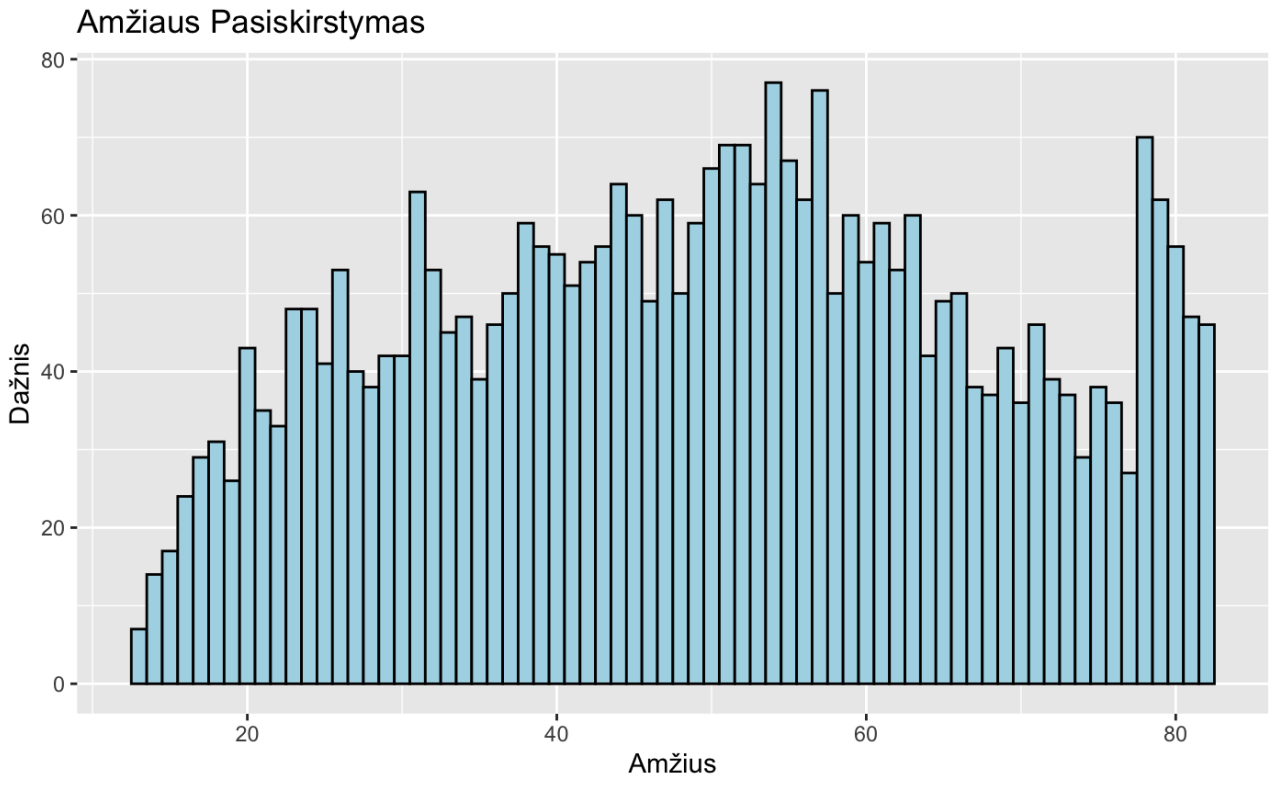
3 pav. *Vidutinio gliukozes kiekio pasiskirstymas pagal insultą*

Stačiakampėje diagramoje *(žr. 3 pav.)* matyti, kad pacientų, kurie yra patyrę insultą vidutinis gliukozės kiekis yra labai plačiai pasiskirstęs. Taip pat matome, kad tarp nepatyrusių insulto pacientų yra nemažai išskirtinių vidutinio gliukozės kiekio reikšmių ir pasiskirstymas yra siauresnis palyginus su patyrusių insultą.



4 pav. *KMI pasiskirstymas*

Histogramoje *(žr. 4 pav.)* matyti, kad pacientų kūno masės indeksas (KMI) dažniausiai pasiskirstęs intervale maždaug nuo 20 iki 35. Taip pat matyti, kad pasiskirstymas šiek tiek pasviręs į dešinę. Yra mažiau, bet pasitaiko didesnių KMI reikšmių (virš 40), kurios sudaro ilgesnę dešinę uodegą.



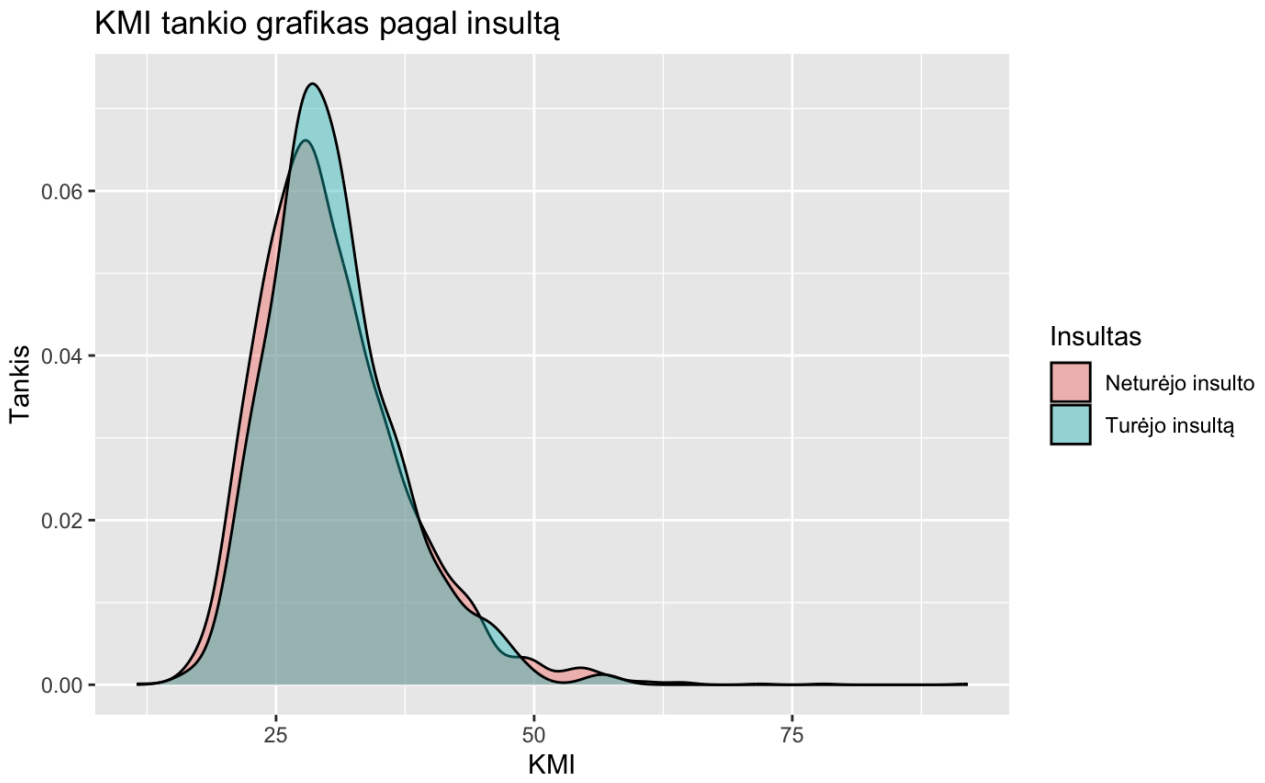
5 pav. *Amžiaus pasiskirstymas*

Iš grafiko *(žr. 5 pav.)* matyti, kad pacientų amžius yra gana plačiai pasiskirstęs. Daugiausiai pacientų yra vidutinio ir vyresnio amžiaus grupėse, maždaug nuo 45 iki 65 metų. Jaunesnių pacientų yra mažiau, todėl šioje amžiaus dalyje dažnis mažesnis. Bendras pasiskirstymas gana tolygus visame amžiaus intervale.



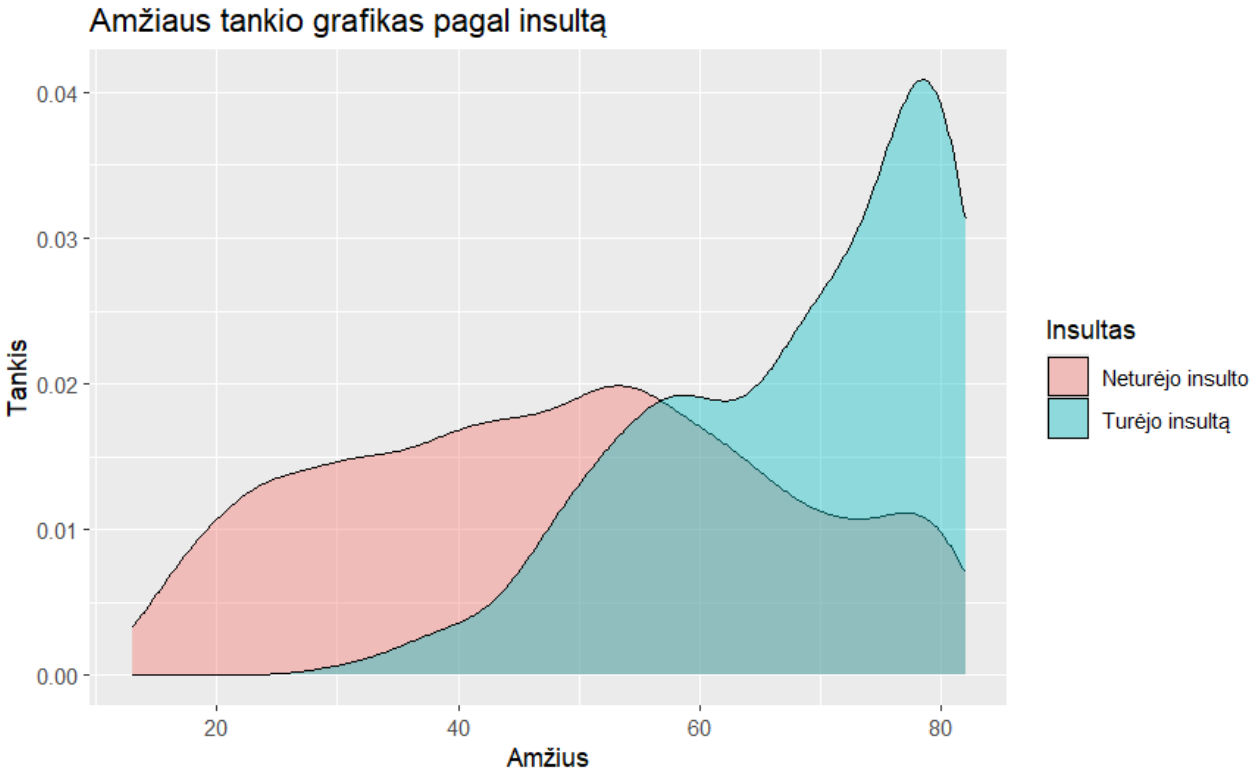
6 pav. *Vidutinio gliukozės kiekio pasiskirstymas*

Matome *(žr. 6 pav.)*, kad vidutinis gliukozės kiekis dažniausiai pasiskirstęs maždaug nuo 70 iki 110. Didžiausia reikšmių koncentracija yra apie 80–100. Taip pat matyti, kad pasiskirstymas yra pasviręs į dešinę. Nors didžioji dalis reikšmių yra mažesnės, pasitaiko ir gerokai didesnių gliukozės kiekio reikšmių (virš 200), kurios sudaro ilgesnę dešinę uodegą.



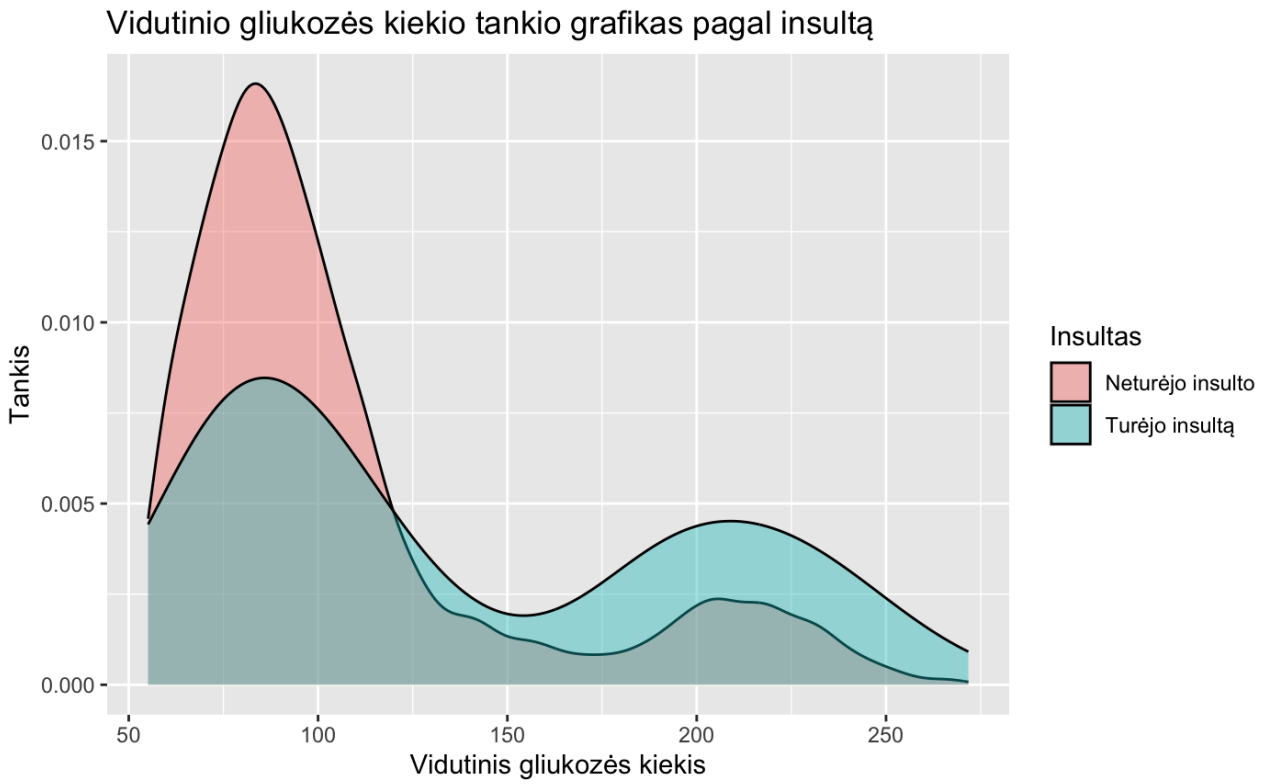
7 pav. *KMI tankis pagal insultą*

KMI tankio grafike *(žr. 7 pav.)* matyti, kad KMI tankis tarp insultą patyrusių ir nepatyrusių pacientų yra gana panašus. Didžiausia reikšmių dalis abiejose grupėse yra apie $25–30$ KMI. Taip pat matyti, kad abiejose grupėse pasitaiko ir didesnių KMI reikšmių, tačiau jos yra retesnės. Bendras tankis tarp grupių labai panašus.



8 pav. *Amžiaus tankio grafikas pagal insultą*

Tankio grafike *(žr. 8 pav.)* matyti, kad insultą patyrusių pacientų amžius dažniau yra didesnis. Didžiausias tankis yra vyresnio amžiaus grupėje apie $60–80$ metų. Tuo tarpu insulto nepatyrę pacientai dažniau yra jaunesnio ir vidutinio amžiaus. Tai gali rodyti, kad insultas dažniau pasitaiko vyresniems žmonėms.



9 pav. *Vidutinio gliukozės kiekio tankis pagal insultą*

Matome *(žr. 9 pav.)*, kad insultą patyrusių pacientų vidutinis gliukozės kiekis dažniau yra didesnis. Nors didžiausia dalis abiejų grupių koncentruojasi ties norma (apie 70–100 mg/dL), insultą patyrusių asmenų grupėje išsiskiria antras tankio šuolis ties 200–230 mg/dL. Tuo tarpu insulto nepatyrę pacientai beveik visais atvejais pasižymi tik mažesniu, norma laikomu gliukozės kiekiu. Tai gali rodyti, kad vidutinis gliukozės kiekis kraujyje įtakos insultui neturės.

3 lentelė. *Kategorinių kintamųjų dažniai duomenų rinkinyje*

Kintamasis	**Kategorija**	**Dažnis**
Lytis	Moteris	2043
	Vyras	1300
Hipertenzija	Neturi	2935
	Turi	408
Širdies liga	Neturi	3137
	Turi	206
Darbo tipas	Valstybinis	514
	Privatus	2200
	Individualus	629
Gyvenamoji vieta	Kaimas	1645
	Miestas	1698
Rūkymo statusas	Metė rūkyti	824
	Niekada nerūkė	1784
	Rūko	735

Matome *(žr. 3 lentelėje)*, kad duomenų rinkinyje moterų yra daugiau nei vyrų, o dauguma tiriamųjų neturi hipertenzijos ir širdies ligų. Darbo tipo pasiskirstymas rodo, kad didžiausia grupė dirba

privačiame sektoriuje, mažiau yra valstybinių darbuotojų ir individualiai dirbančių. Taip pat matome, kad gyvenamoji vieta yra gana subalansuota. Rūkymo statuso atžvilgiu dauguma niekada nerūkė. Kai kurie kintamieji tokie, kaip hipertenzija ar širdies ligos, yra stipriai nesubalansuoti, todėl svarbu į tai atsižvelgti dėl nesubalansuotų klasių poveikio rezultatams.

4 lentelė. *Kategorinių kintamųjų pasiskirstymas pagal insulto reikšmę*

Kintamasis	**Kategorija**	**Nepatyrę insulto**	**Patyrę insultą**
Lytis	Moteris	1938	105
	Vyras	1225	75
Santuokinė padėtis	Ne	724	20
	Taip	2439	160
Hipertenzija	Neturi	2812	123
	Turi	351	57
Širdies liga	Neturi	2993	144
	Turi	170	36
Darbo tipas	Valstybinis	491	23
	Privatus	2091	109
	Individualus	581	48
Gyvenamoji vieta	Kaimas	1559	86
	Miestas	1604	94
Rūkymo statusas	Metė rūkyti	767	57
	Niekada nerūkė	1700	84
	Rūko	696	39

Matome *(žr. 4 lentelėje)*, kad insulto nepatyrusių pacientų yra gerokai daugiau nei patyrusių. Tarp insultą patyrusių pacientų santykinai dažniau pasitaiko hipertenzija ir širdies ligos nei tarp insulto nepatyrusių. Pagal lytį insultą patyrusių grupėje taip pat šiek tiek daugiau moterų nei vyrų, panaši tendencija matoma ir tarp insulto nepatyrusių. Santuokinės padėties atžvilgiu abiejose grupėse dauguma pacientų yra susituokę. Darbo tipo, gyvenamosios vietos ir rūkymo statuso pasiskirstymas tarp patyrusių ir nepatyrusių insulto yra gana panašus, nors daugiausia pacientų dirba privačiame sektoriuje, gyvena tiek mieste, tiek kaime panašiai, o didžiausia dalis niekada nerūkė.

4. Modeliai

Prieš sudarant modelius, turimi duomenys buvo suskaidyti į mokymo (80%) ir testavimo (20%) aibes. Duomenų skaidymas atliktas taikant sluoksninę imtį, kad būtų išlaikytas panašus priklausomojo kintamojo pasiskirstymas abiejose aibėse.

5 lentelė. *Priklausomo kintamojo (insulto) pasiskirstymas mokymo aibėje*

Nepatyrę insulto	Patyrę insultą
2530	144

6 lentelė. *Priklausomo kintamojo (insulto) pasiskirstymas testavimo aibėje*

Nepatyrę insulto	Patyrę insultą
633	36

Matome *(žr. 6 ir 7 lentelėse)*, kad priklausomo kintamojo pasiskirstymas mokymo ir testavimo aibėse yra proporcingas, todėl abi aibės yra reprezentatyvios ir tinkamos modelių mokymui bei vertinimui.

4.1. *logit* modelis

Pirmasis modelis, kurį išbandėme, buvo logistinės (*logit*) regresijos modelis, kuris matematiškai išreiškiamas taip:

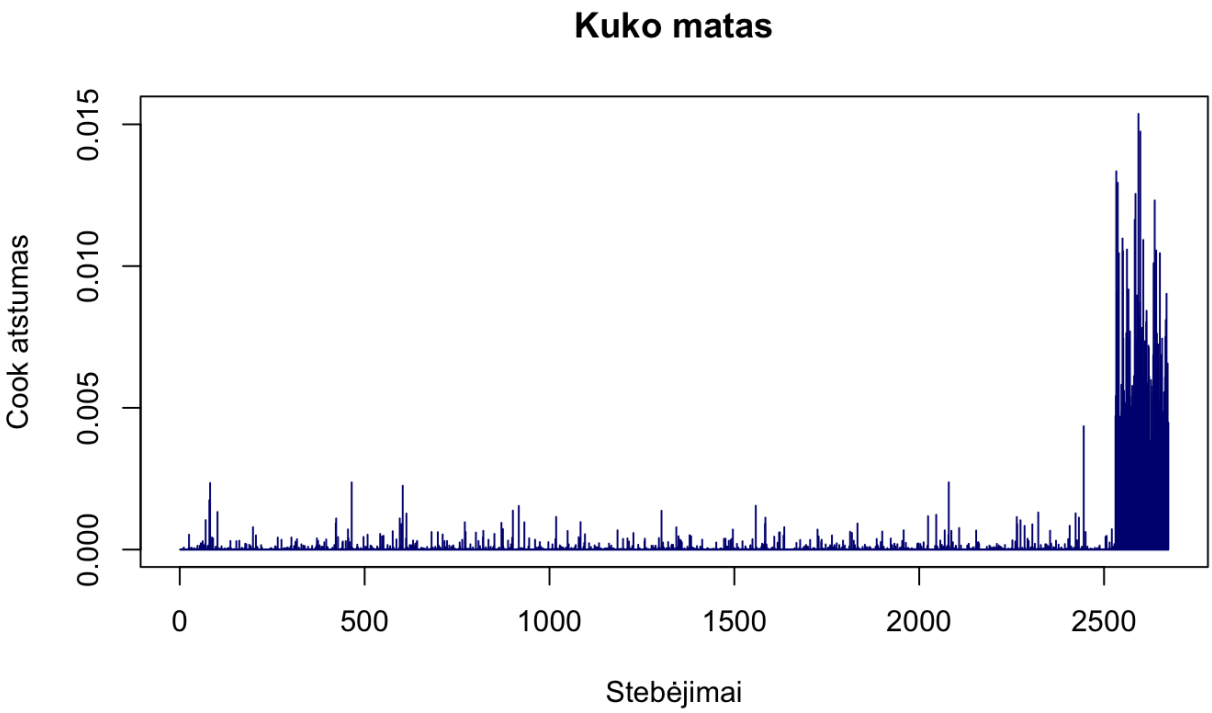
$$\textit{logit}(X) = \ln\left(\frac{\pi(X)}{1-\pi(X)}\right) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_m X_m \tag{1}$$

čia:

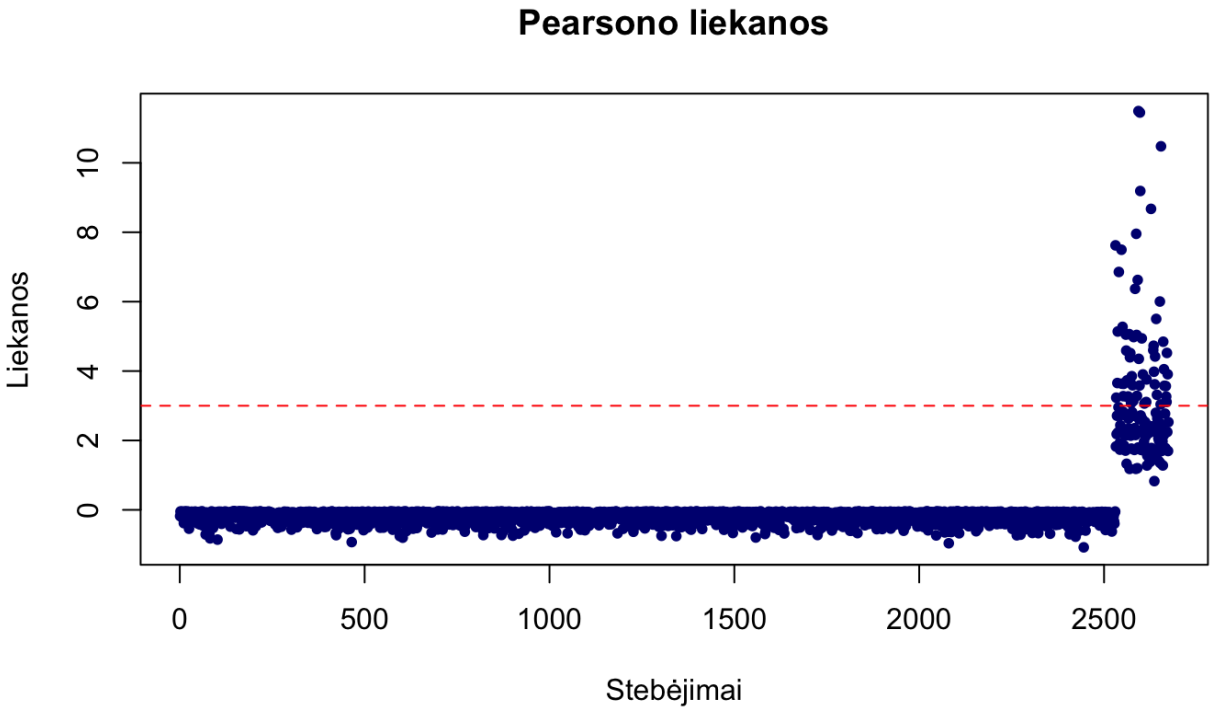
$$\pi(X) = \frac{\exp(\beta_0 + \beta_1 X_1 + \cdots + \beta_m X_m)}{1 + \exp(\beta_0 + \beta_1 X_1 + \cdots + \beta_m X_m)} = P(Y = 1 \mid X)$$

β_i - koeficientas prie kovariantės X_i.

Į modelį buvo įtrauktos visos galimos kovariantes. Nors logistinės regresijos modelis nėra jautrus išskirtims, bet vis tiek buvo patikrinta, ar yra išskirčių su „Cook“ matu ir „Pearson“ liekanomis.



10 pav. *„Cook“ mato grafikas*



11 pav. *„Pearson“ liekanų grafikas*

„Cook“ matas parodė *(žr. 10 pav.)*, kad nė vienas stebėjimas neviršijo ribinės 1 reikšmės. Vis dėl

to, analizuojant Pirsono liekanas *(žr. 11 pav.)* nustatyti 58 stebėjimai, kurių reikšmė viršijo 3, todėl šie stebėjimai buvo laikomi išskirtimis ir pašalinti iš mokymo aibės.

Kadangi multikolinearumas padidina koeficientų standartines paklaidas, kai keli nepriklausomi kintamieji yra stipriai tarpusavyje susiję, todėl buvo patikrinta, ar tarp modelio kovariančių nėra multikolinearumo.

7 lentelė. *Multikolinearumas*

Kintamasis	**GVIF**	**Laisvės laipsniai**	$GVIF^{1/(2\cdot Df)}$
Amžius	$1{,}2053$	1	$1{,}0979$
KMI	$1{,}2046$	1	$1{,}0976$
Vidutinis gliukozės kiekis	$1{,}0870$	1	$1{,}0426$
Lytis	$1{,}0884$	1	$1{,}0432$
Hipertenzija	$1{,}0200$	1	$1{,}0100$
Širdies ligos	$1{,}0549$	1	$1{,}0271$
Santuoka	$1{,}0196$	1	$1{,}0098$
Darbo tipas	$1{,}0553$	2	$1{,}0135$
Gyvenamosios vietos tipas	$1{,}0155$	1	$1{,}0077$
Rūkymo statusas	$1{,}0948$	2	$1{,}0229$

Nustatyta *(žr. 7 lentelėje)*, kad visų kintamųjų GVIF reikšmės neviršijo 4, todėl reikšmingo multikolinearumo tarp kintamųjų nepastebėta.

Atlikus pradinę duomenų analizę buvo nustatyta nesubalansuotų klasių problema. Šiai problemai spręsti pasirinktas klasifikavimo slenksčio keitimas. Remiantis *Youden* indeksu nustatyta, kad optimalus klasifikavimo slenkstis apytiksliai yra $0{,}033$.

Toliau modelis buvo pritaikytas ir testavimo aibės duomenims. Naudojant gautą optimalų slenkstį gauname tokią klasifikavimo lentelę su testavimo duomenimis:

8 lentelė. *Klasifikavimo lentelė logit modeliui testavimo aibei*

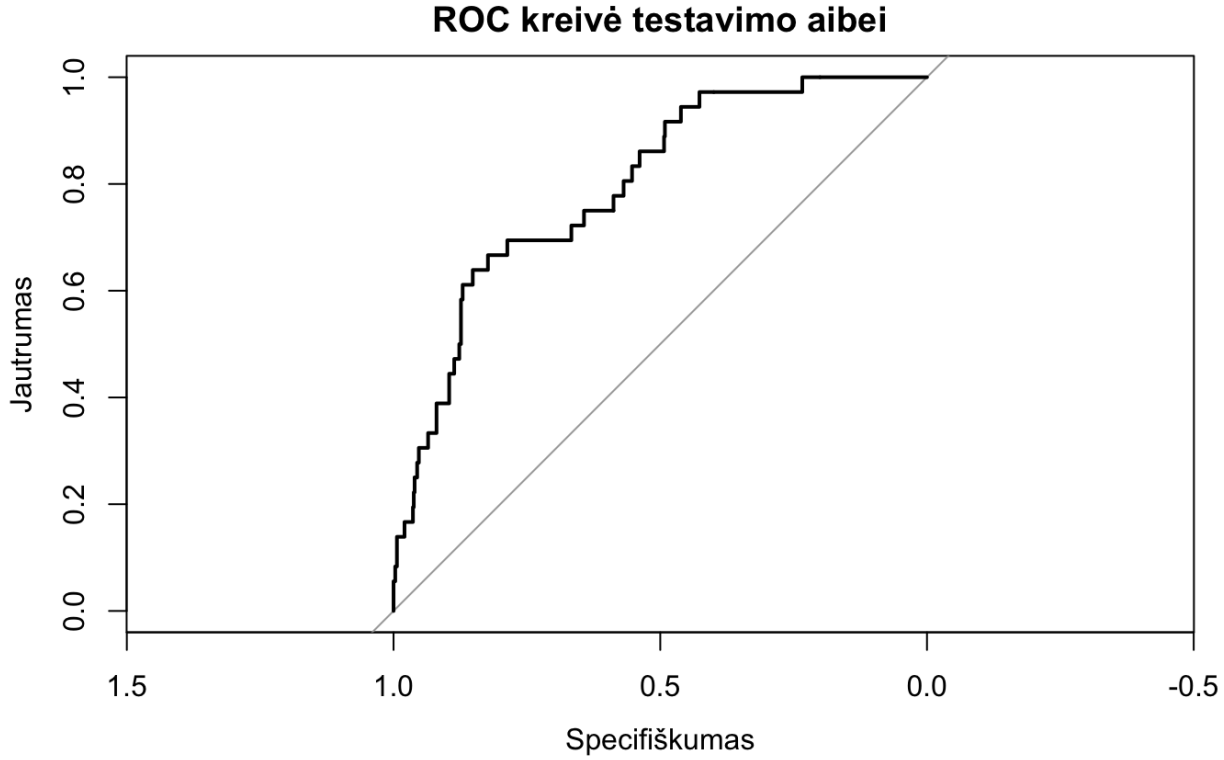
	Prognozuota	
Tikra	Patyrę insultą	Nepatyrę insulto
Patyrę insultą	23	13
Nepatyrę insulto	104	529

Iš klasifikavimo lentelės *(žr. 8 lentelėje)* gauname:

9 lentelė. *Klasifikavimo tikslumas logit modeliui*

Tikslumo matas	Reikšmė
Jautrumas	$\approx 0{,}64$
Specifiškumas	$\approx 0{,}84$
Tikslumas	$\approx 0{,}83$
F1 balas	$\approx 0{,}28$

Matyti *(žr. 8 ir 9 lentelėse)*, kad modelis gerai atpažįsta pacientus be insulto (specifiškumas - $0{,}84$), tačiau silpnai prognozuoja insulto atvejus (jautrumas - $0{,}64$). Kadangi mums svarbiausia teisingai prognozuoti patyrusiųjų insultą klasę, šie rezultatai rodo, kad modelio gebėjimas identifikuoti insultą yra ribotas ir reikėtų tobulinti prognozavimo tikslumą šiai grupei.



12 pav. *ROC kreivė logit modeliui testavimo aibei*

Grafike *(žr. 12 pav.)* matyti, kad ROC kreivė yra aukščiau už atsitiktinio klasifikavimo įstrižainę, todėl galima teigti, kad logit modelis testavimo aibėje geba atskirti klases geriau nei atsitiktinai.

4.2. *probit* modelis

Antrasis nagrinėtas modelis yra *probit* modelis. Jis matematiškai išreiškiamas taip:

$$P(Y = 1 \mid X) = \Phi(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_k X_k) \tag{2}$$

čia:

$\Phi(\cdot)$ - standartinio normaliojo dydžio pasiskirstymo funkcija.

Kadangi sudarant pirmąjį modelį buvo pašalintos išskirtys iš mokymo aibės ir patikrintas multikolinearumas tarp kovariančių, ši analizė tolesniems modeliams yra nebekartojama.

Taip pat pagal *Youden* indeksą surandame optimalų slenkstį, kuris apytiksliai yra $0{,}041$. Ir pasinaudoję juo gauname tokią klasifikavimo lentelę:

10 lentelė. *Klasifikavimo lentelė probit modeliui testavimo aibei*

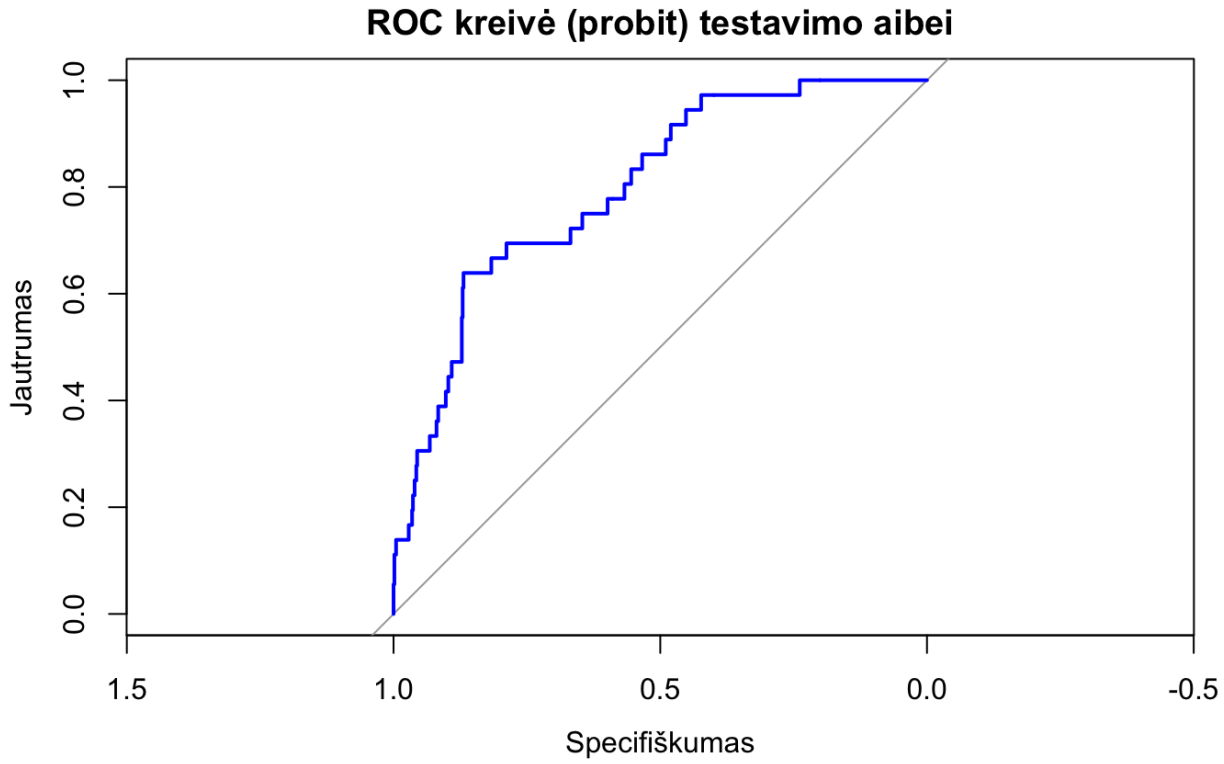
	Prognozuota	
Tikra	Patyrę insultą	Nepatyrę insulto
Patyrę insultą	23	13
Nepatyrę insulto	94	539

Iš klasifikavimo lentelės *(žr. 10 lentelėje)* gauname:

11 lentelė. *Klasifikavimo tikslumas*

Tikslumo matas	**Reikšmė**
Jautrumas	$\approx 0{,}64$
Specifiškumas	$\approx 0{,}85$
Tikslumas	$\approx 0{,}84$
F1 balas	$\approx 0{,}30$

Matome *(žr. 9 ir 10 lentelėse)*, kad *probit* modelio jautrumo rezultatai sutampa su *logit* modelio, bet visi kiti gauti tikslumo rodikliai su *probit* modeliu yra geresni.



13 pav. *ROC kreivė probit modeliui testavimo aibei*

Matome, *(žr. 13 pav.)*, kad *probit* modelio ROC kreivė yra aukščiau už atsitiktinio klasifikavimo liniją, todėl galima teigti, kad modelis testavimo aibėje geba atskirti klases geriau nei atsitiktinai.

4.3. *cloglog* modelis

Paskutinis modelis kurį nagrinėjome yra *cloglog* modelis. Jo matematinė formulė:

$$\textit{cloglog}(\pi) = \ln\Big(-\ln(1-\pi)\Big) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p \tag{3}$$

čia:

$$\pi = P(Y = 1 \mid X) = 1 - \exp\Big(-\exp(\beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p)\Big)$$

Pasinaudoję *Youden* indeksu surandame optimalų slenkstį, kuris apytiksliai yra $0{,}033$. Ir pasinaudoję juo gauname tokią klasifikavimo lentelę:

12 lentelė. *Klasifikavimo lentelė cloglog testavimo aibei*

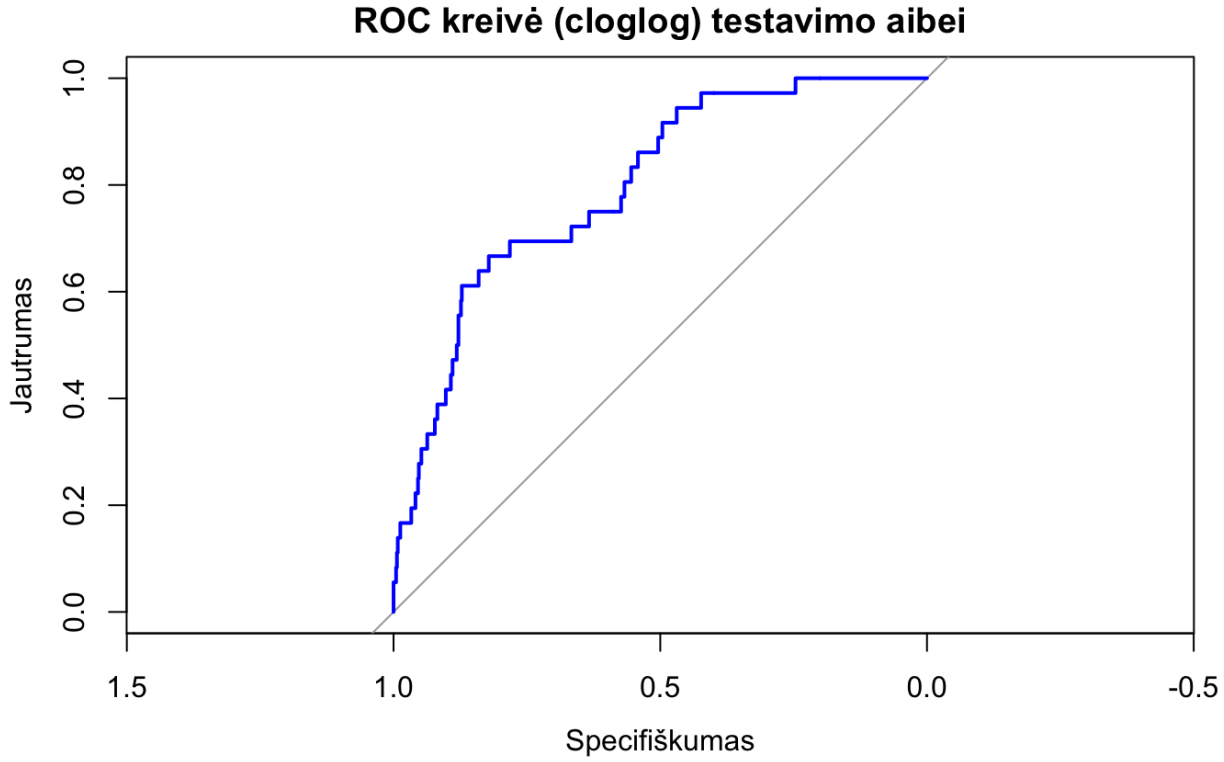
	Prognozuota	
Tikra	Patyrę insultą	Nepatyrę insulto
Patyrę insultą	23	13
Nepatyrę insulto	104	529

Iš klasifikavimo lentelės *(žr. 12 lentelėje)* gauname:

13 lentelė. *Klasifikavimo tikslumas*

Tikslumo matas	Reikšmė
Jautrumas	$\approx 0{,}64$
Specifiškumas	$\approx 0{,}84$
Tikslumas	$\approx 0{,}83$
F1 balas	$\approx 0{,}28$

Matome *(žr. 12 ir 13 lentelėse)*, kad *cloglog* tikslumo rezultatai sutampa su *logit* modelio, bet, jei žiūrėtume į specifiškumą, tikslumą ir F1 balą, yra prastesni už *probit* modelio rezultatus.



14 pav. *ROC kreivė cloglog modeliui testavimo aibei*

Grarfike *(žr. 14 pav.)* matyti, kad ROC kreivė yra aukščiau už atsitiktinio klasifikavimo įstrižainę, todėl galima teigti, kad cloglog modelis testavimo aibėje geba atskirti klases geriau nei atsitiktinai.

5. Modelio tobulinimas

Nagrinėjant medicininius duomenis svarbu kuo tiksliau nustatyti atvejus, kai pacientas serga, todėl renkantis geriausią modelį didžiausias dėmesys bus skiriamas jautrumo (angl. sensitivity) rodikliui ir klasifikavimo lentelei. Iš 5 skyriaus rezultatų matyti, kad visi modeliai vienodu tikslumu teisingai prognozuoja insultą. Žiūrint į kitus klasifikavimo tikslumo rodiklius paaiškėja, kad tikslesnis modelis yra *probit*, todėl jį laikome, tinkamiausiu mūsų duomenims.

Atliksime pažingsninę regresiją siekdami atrinkti reikšmingiausias kovariantes ir supaprastinti modelį. Vertindami gautą modelį atsižvelgsime į klasifikavimo tikslumo rodiklius.

Atlikus pažingsninę regresiją gavome tokią modelio išraišką:

$$P(Y = 1 \mid X) = \Phi\Big(\beta_0 + \beta_1 \text{age} + \beta_2 \text{avg_glucose_level} + \beta_3 \text{hypertension} + \beta_4 \text{heart_disease} + \beta_5 \text{work_type}\Big) \tag{4}$$

Taip pat pateikiame ir klasifikavimo lentelę su tikslumo rodikliais testavimo aibei:

14 lentelė. *Klasifikavimo lentelė probit modeliui testavimo aibei*

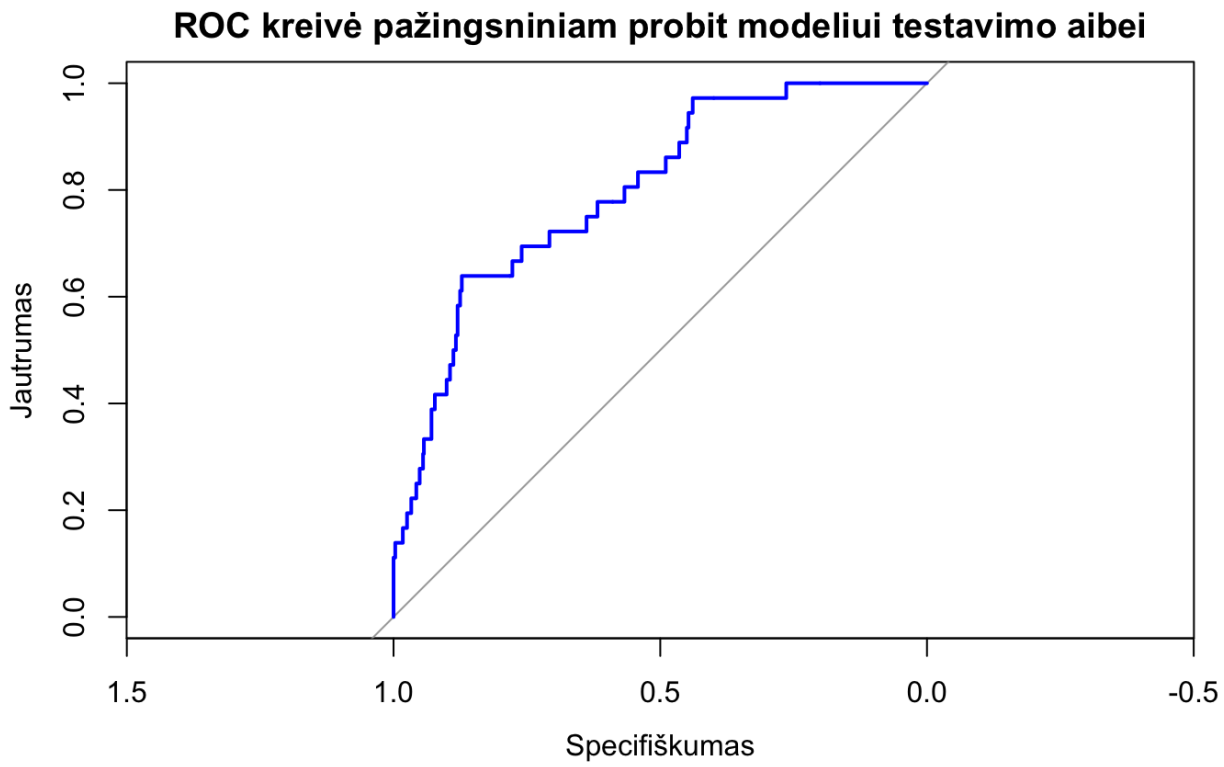
	Prognozuota	
Tikra	Patyrę insultą	Nepatyrę insulto
Patyrę insultą	23	13
Nepatyrę insulto	98	535

Iš klasifikavimo lentelės *(žr. 14 lentelėje)* gauname:

15 lentelė. *Klasifikavimo tikslumas*

Tikslumo matas	**Reikšmė**
Jautrumas	$\approx 0{,}64$
Specifiškumas	$\approx 0{,}85$
Tikslumas	$\approx 0{,}83$
F1 balas	$\approx 0{,}29$

Lentelėse matyti *(žr. 14 ir 15 lentelėse)*, kad klasifikavimo tikslumo rezultatai šiek tiek suprastėjo nei pradiniame *probit* modelyje su visomis kovariantėmis, todėl ir naudosime pradinį modelį.



15 pav. *ROC kreivė pažingsniniam probit modeliui testavimo aibei*

Matome *(žr. 15 pav.)*, kad ROC kreivė yra aukščiau už atsitiktinio klasifikavimo įstrižainę, todėl galima teigti, kad pažingsninis *probit* modelis testavimo aibėje geba atskirti klases geriau nei atsitiktinai.

5.1. *Probit* modelis su vidutinio gliukozės lygio ir širdies ligų sąveika

Į modelį buvo įtraukta vidutinio gliukozės lygio kraujyje ir turimų širdies ligų sąveika (Average glucose level $\times$ heart disease), nes mediciniškai pagrįsta manyti, kad gliukozės poveikis insulto tikimybei gali būti kitoks pacientams, kurie turi širdies ligų, negu tiems, kurie jų neturi.

Modelio su sąveikomis išraiška:

$$P(Y = 1 \mid X) = \Phi(\eta), \tag{5}$$

$$\begin{aligned}\eta = \beta_0 &+ \beta_1 \,\text{heart_disease} + \beta_2 \,\text{avg_glucose_level} + \beta_3 \,(\text{heart_disease} \times \text{avg_glucose_level}) \\ &+ \beta_4 \,\text{bmi} + \beta_5 \,\text{gender} + \beta_6 \,\text{age} + \beta_7 \,\text{hypertension} \\ &+ \beta_8 \,\text{ever_married} + \beta_9 \,\text{work_type} + \beta_{10} \,\text{Residence_type} + \beta_{11} \,\text{smoking_status}.\end{aligned}$$

Pasinaudoję *Youden* indeksu surandame optimalų slenkstį, kuris yra 0,01070305, ir gauname tokią klasifikavimo lentelę:

16 lentelė. *Klasifikavimo lentelė testavimo aibei (probit su sąveika)*

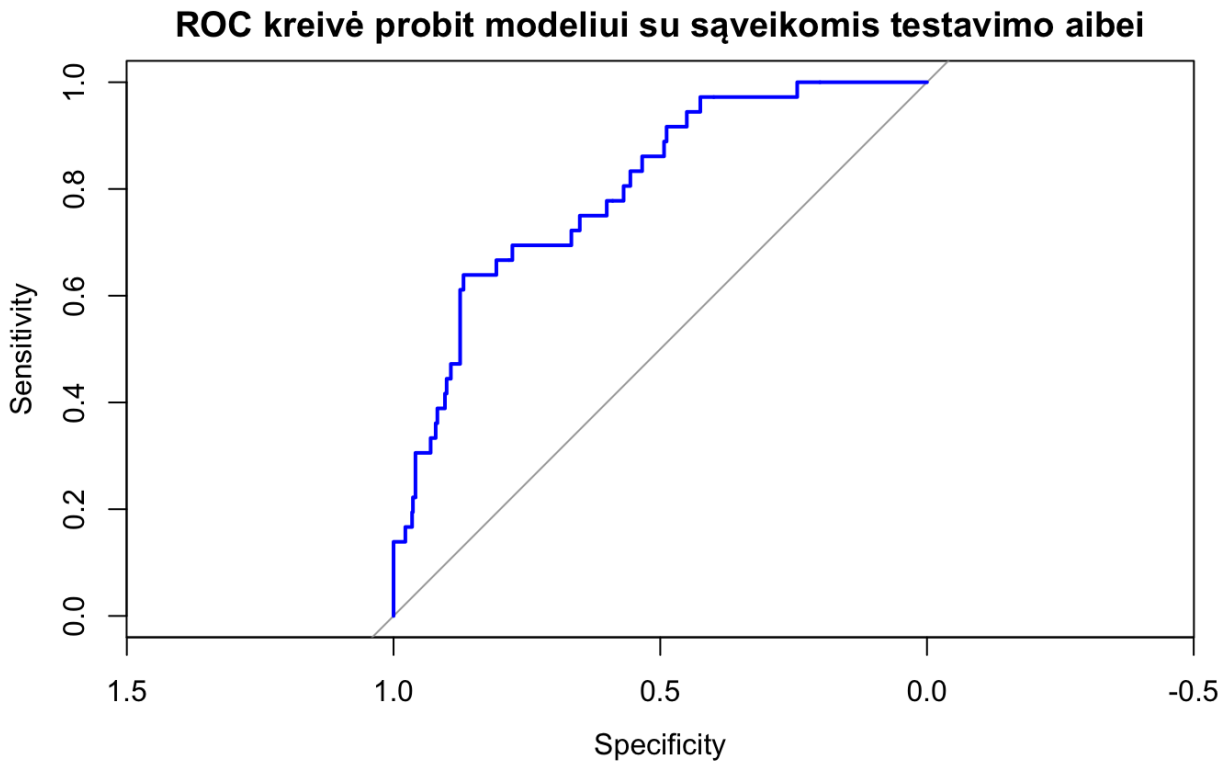
	Prognozuota	
Tikra	Patyrę insultą	Nepatyrę insulto
Patyrę insultą	23	13
Nepatyrę insulto	94	539

Iš klasifikavimo lentelės *(žr. 16)* gauname:

17 lentelė. *Klasifikavimo tikslumas*

Tikslumo matas	**Reikšmė**
Jautrumas	$\approx 0{,}64$
Specifiškumas	$\approx 0{,}85$
Tikslumas	$\approx 0{,}84$
F1 balas	$\approx 0{,}30$

Iš lentelės *(žr. 16 ir 17 lentelėse)* galima matyti, kad naudojant *probit* modelį su sąveika, gaunami rezultatai yra identiški pradinio modelio rezultatams.



16 pav. *ROC kreivė testavimo aibei (probit) modeliui su sąveika*

Grafike *(žr. 16 pav.)* matyti, kad kreivė yra aukščiau atsitiktinės klasifikacijos linijos, todėl galima teigti, kad modelis pasižymi pakankamai geru gebėjimu klasifikuoti pacientus į patyrusių ir nepatyrusių insultą klases.

Kadangi buvo nustatyta, kad *probit* modelis su sąveika klasifikavimo rezultatų nepagerina, tolimesniam tyrimui buvo naudojamas pradinis modelis be sąveikos.

6. Geriausio modelio interpretacija

6.1. 95 % pasikliovimo intervalai (*probit* modelis)

Toliau apskaičiavome *probit* modelio koeficientų 95 % pasikliovimo intervalus testavimo duomenims.

18 lentelė. *95 % pasikliovimo intervalai (probit modelis)*

Kintamasis	**2.5 %**	**97.5 %**
(Intercept)	$-10{,}384$	$-6{,}786$
Amžius	$0{,}061$	$0{,}098$
Kūno masės indeksas (BMI)	$-0{,}008$	$0{,}037$
Vidutinis gliukozės kiekis	$0{,}002$	$0{,}006$
Lytis (vyras)	$-0{,}211$	$0{,}318$
Hipertenzija (taip)	$0{,}102$	$0{,}634$
Širdies ligos (taip)	$0{,}110$	$0{,}717$
Santuokos statusas (ne)	$-0{,}470$	$0{,}424$
Darbo tipas (privatus)	$-0{,}012$	$0{,}888$
Darbo tipas (sav. įmonė)	$-0{,}387$	$0{,}569$
Gyvenamoji vieta (miestas)	$-0{,}413$	$0{,}098$
Rūkymo statusas (nerūko)	$-0{,}217$	$0{,}356$
Rūkymo statusas (rūko)	$-0{,}374$	$0{,}443$

Matome *(žr. 18 lentelėje)*, kad statistiškai reikšmingi yra amžius, vidutinis gliukozės kiekis, hipertenzija ir turimos širdies ligos. Visi jie padidina insulto tikimybę. Kitų kintamųjų pasikliovimo intervalai apima nulį, todėl statistiškai jie nėra reikšmingi. Amžiaus ir vidutinio gliukozės kiekio intervalai yra santykinai siauri, todėl šių kintamųjų poveikio įverčiai laikomi tikslesniais.

6.2. Marginalieji efektai (*probit* modelis)

Probit modelio koeficientų poveikis priklausomo kintamojo tikimybei aiškinamas naudojant marginaliuosius efektus. Kadangi *probit* modelio koeficientai tiesiogiai neparodo tikimybės pokyčio, interpretacijai naudojamos išvestinės, kurios leidžia įvertinti, kaip keičiasi įvykio tikimybė padidinus aiškinamąjį kintamąjį vienu vienetu, laikant kitus kintamuosius pastoviais. Interpretacijai naudojama bendroji *probit* modelio išvestinės forma.

$$\frac{\partial E(y_i \mid x_i)}{\partial avg_glucose_level_i} = \phi(x_i'\beta)\beta_{glucose}. \tag{6}$$

19 lentelė. *Vidutiniai marginalieji efektai (probit modelis)*

Kintamasis	**Marginalus efektas**
(Intercept)	$-0{,}40304$
Amžius	$0{,}00372$
Kūno masės indeksas	$0{,}00070$
Vidutinis gliukozės kiekis	$0{,}00020$
Lytis (vyras)	$0{,}00258$
Hipertenzija (taip)	$0{,}01753$
Širdies ligos (taip)	$0{,}01978$
Santuokos statusas (taip)	$0{,}00027$
Darbo tipas (privatus)	$0{,}01967$
Darbo tipas (sav. įmonė)	$0{,}00343$
Gyvenamoji vieta (miestas)	$-0{,}00742$
Rūkymo statusas (nerūko)	$0{,}00321$
Rūkymo statusas (rūko)	$0{,}00205$

Matome *(žr. 19 lentelėje)*, kad didžiausią teigiamą poveikį insulto tikimybei šiame modelyje turi širdies ligos, darbo tipas (privatus) ir hipertenzija. Širdies ligos marginalus efektas rodo, kad asmenims, sergantiems širdies liga, insulto tikimybė vidutiniškai padidėja apie 1,98 %, kai kiti kintamieji laikomi pastoviais. Hipertenzijos buvimas padidina insulto tikimybę apie 1,75 % ir darbas privačiame sektoriuje apie 1,97 %. Kiti kintamieji ne taip stipriai didina insulto riziką, o ją mažina tik miesto gyvenamoji vieta.

Išvados

1. Pradinė duomenų analizė parodė, kad didžiausią ryšį su insulto tikimybe turi amžius. Didėjant amžiui insulto rizika gali didėti.

2. Multikolinearumo analizė parodė, kad tarp į modelius įtrauktų kovariančių nėra stiprių tarpusavio priklausomybių, tačiau buvo rasta išskirčių, kurias pašalinome todėl tapo kintamieji tinkami regresinei analizei.

3. Sudaryti *logit*, *probit* ir *cloglog* modeliai parodė panašų gebėjimą prognozuoti insulto tikimybę, tačiau visais atvejais geriau buvo atpažįstami pacientai be insulto nei insultą patyrę pacientai.

4. Palyginus modelius nustatyta, kad *probit* modelis pasižymėjo geriausiais klasifikavimo rodikliais, nes jo specifiškumas, bendras tikslumas ir F1 balas buvo geresni nei *logit* ir *cloglog* modelių.

5. Pažingsninė regresija *probit* modeliui prognozavimo kokybės nepagerino, todėl galutiniam vertinimui buvo pasirinktas pradinis *probit* modelis.

6. Į modelį įtraukus vidutinio gliukozės kiekio ir širdies ligų sąveiką, rezultatai nepasikeitė, todėl geresniu laikomas pradinis *probit*.

7. Pasikliovimo intervalų analizė parodė, kad statistiškai reikšmingi insulto rizikos veiksniai yra amžius, hipertenzija, vidutinis gliukozės kiekis ir esamos širdies ligos.

8. Vidutinių marginaliųjų efektų interpretacija parodė, kad didžiausią teigiamą poveikį insulto tikimybei *probit* modelyje turi širdies ligos, darbas privačiame sektoriuje ir hipertenzija.

Rekomendacijos

1. Vertinant insulto riziką praktikoje, ypatingą dėmesį rekomenduojama skirti vyresnio amžiaus pacientams, turintiems padidėjusį gliukozės kiekį kraujyje ir hipertenziją.
2. Ateityje modelį tikslinga papildyti papildomais klinikiniais rodikliais, kurie galėtų pagerinti insultą patyrusių pacientų atpažinimą.
3. Praktiniam taikymui rekomenduojama modelį tikrinti su naujais pacientų duomenimis, kad būtų įvertintas jo stabilumas ir pritaikomumas skirtingose populiacijose.

Literatūros sąrašas

[1] *Stroke Prediction Dataset*. Kaggle. Prieiga per internetą: `https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset`

[2] Čekanavičius, V., Murauskas, G. (2014). *Taikomoji regresinė analizė socialiniuose tyrimuose*. Vilnius.

[3] Doc., Dr. Rūta Levulienė (2025). *Regresinė analizė: paskaitų teorinės medžiagos skaidrės*. Vilniaus universitetas, Duomenų mokslo studijų programa.

[4] OpenAI. (2024). *ChatGPT (GPT-4)* [Dirbtinio intelekto modelis]. Naudota darbo aprašymo lentelėms formatuoti, kodo bei rašybos klaidų taisymui.

Priedas A: Pilnas R kodas

```
library(dplyr)
library(ggplot2)
library(pROC)
library(car)
library(caret)

df <- read.csv("healthcare-dataset-stroke-data.csv", header = TRUE, sep = ",")

any(duplicated(df$id))
df <- df %>% select(-id)

df <- na.omit(df)

df <- df %>%
  filter(bmi!="N/A")

df <- df %>%
  filter(smoking_status != "Unknown") # 30% reiksmiu buvo is 5000 irasu

df <- df %>%
  filter(gender != "Other")

df <- df %>%
  filter(!work_type %in% c("Never_worked", "children"))

lapply(df[, c("gender", "hypertension", "heart_disease", "ever_married",
↪  "work_type", "Residence_type", "smoking_status", "stroke")], unique)

any(is.na(df[, c("age", "bmi", "avg_glucose_level")]))

sapply(df[, c("age", "bmi", "avg_glucose_level")], function(x) c(min = min(x, na.rm
↪  = TRUE),
                                                                 max = max(x, na.rm =
                                                                 ↪  TRUE)))

table(df$stroke)

df$bmi <- as.numeric(df$bmi)

boxplot(bmi ~ factor(stroke,
                     levels = c(0,1),
```

```
                    labels = c("Neturėjo insulto", "Turėjo insultą")),
        data = df,
        main = "KMI pasiskirstymas pagal insultą",
        xlab = "Insultas",
        ylab = "KMI",
        col = c("lightblue","lightgreen"))

boxplot(age ~ factor(stroke,
                    levels = c(0,1),
                    labels = c("Neturėjo insulto", "Turėjo insultą")),
        data = df,
        main = "Amžiaus pasiskirstymas pagal insultą",
        xlab = "Insultas",
        ylab = "Metai",
        col = c("lightblue","lightgreen"))

boxplot(avg_glucose_level ~ factor(stroke,
                    levels = c(0,1),
                    labels = c("Neturėjo insulto", "Turėjo insultą")),
        data = df,
        main = "Vidutinio gliukozės kiekio pasiskirstymas pagal insultą",
        xlab = "Insultas",
        ylab = "Vidutinis gliukozės kiekis",
        col = c("lightblue","lightgreen"))

ggplot(df, aes(x=bmi)) +
  geom_histogram(binwidth=2, fill="lightgreen", color="black") +
  labs(title="KMI pasiskirstymas", x="KMI", y="Dažnis")

ggplot(df, aes(x=age)) +
  geom_histogram(binwidth=1, fill="lightblue", color="black") +
  labs(title="Amžiaus pasiskirstymas", x="Amžius", y="Dažnis")

ggplot(df, aes(x=avg_glucose_level)) +
  geom_histogram(binwidth=2, fill="lightpink", color="black") +
  labs(title="Vidutinis gliukozės kiekio pasiskirstymas", x="Vidutinis gliukozės
  ↪  kiekis", y="Dažnis")

ggplot(df, aes(x = age, fill = factor(stroke,
                    levels = c(0,1),
                    labels = c("Neturėjo insulto", "Turėjo insultą")))) +
  geom_density(alpha = 0.4) +
  labs(title = "Amžiaus tankio grafikas pagal insultą",
       fill = "Insulas", x="Amžius", y="Tankis")
```

```r
ggplot(df, aes(x = avg_glucose_level, fill = factor(stroke,
                     levels = c(0,1),
                     labels = c("Neturėjo insulto", "Turėjo insultą")))) +
  geom_density(alpha = 0.4) +
  labs(title = "Vidutinio gliukozės kiekio tankio grafikas pagal insultą",
       fill = "Insultas", x="Vidutinis gliukozės kiekis", y="Tankis")

ggplot(df, aes(x = bmi, fill = factor(stroke,
                     levels = c(0,1),
                     labels = c("Neturėjo insulto", "Turėjo insultą")))) +
  geom_density(alpha = 0.4) +
  labs(title = "KMI tankio grafikas pagal insultą",
       fill = "Insultas", x="KMI", y="Tankis")

lapply(df[, c("gender","hypertension","heart_disease","work_type", "Residence_type",
↪  "smoking_status")], table)

table(df$gender, df$stroke)
table(df$ever_married, df$stroke)
table(df$hypertension, df$stroke)
table(df$heart_disease, df$stroke)
table(df$work_type, df$stroke)
table(df$Residence_type, df$stroke)
table(df$smoking_status, df$stroke)

df$gender <- factor(df$gender)
df$ever_married <- factor(df$ever_married)
df$hypertension <- factor(df$hypertension)
df$heart_disease <- factor(df$heart_disease)
df$work_type <- factor(df$work_type)
df$Residence_type <- factor(df$Residence_type)
df$smoking_status <- factor(df$smoking_status)

df0 <- df[df$stroke == 0, ]
df1 <- df[df$stroke == 1, ]

set.seed(8)

train0_index <- sample(1:nrow(df0), 0.8*nrow(df0))
train1_index <- sample(1:nrow(df1), 0.8*nrow(df1))

train <- rbind(df0[train0_index, ], df1[train1_index, ])
test <- rbind(df0[-train0_index, ], df1[-train1_index, ])
```

```r
table(train$stroke)
table(test$stroke)

model <- glm(stroke ~ age + bmi + avg_glucose_level + gender + hypertension +
↪  heart_disease + ever_married + work_type + Residence_type + smoking_status,
            data = train,
            family = binomial(link = "logit"))
summary(model)

cooksd <- cooks.distance(model)

outliers <- cooksd[cooksd > 1]
outliers

plot(cooksd, type="h", col="navy", main="Kuko matas", xlab="Stebėjimai", ylab="Cook
↪  atstumas")

pearson_resid <- residuals(model, type = "pearson")

plot(pearson_resid, pch=20, col="navy", main="Pearsono liekanos", xlab="Stebėjimai",
↪  ylab="Liekanos")
abline(h=c(-3,3), col="red", lty=2)

isskirtys_pearson <- which(abs(pearson_resid) > 3)
train[isskirtys_pearson, ]

train <- train[-isskirtys_pearson, ]

model <- glm(stroke ~ age + bmi + avg_glucose_level + gender + hypertension +
↪  heart_disease + ever_married + work_type + Residence_type + smoking_status,
                        data = train, family = binomial(link = "logit"))

summary(model)

vif <- vif(model)
vif

probs <- predict(model, newdata = train, type = "response")
rocobj <- roc(train$stroke, probs)

plot(rocobj, main = "ROC kreivė mokymo aibei", xlab="Specifiškumas",
↪  ylab="Jautrumas")
```

```r
optimal_cutoff <- as.numeric(coords(rocobj, "best", best.method = "youden", ret =
↪  "threshold"))
optimal_cutoff


probs <- predict(model, newdata = test, type = "response")

rocobj <- roc(test$stroke, probs)
plot(rocobj, main = "ROC kreivė testavimo aibei", xlab="Specifiškumas",
↪  ylab="Jautrumas")

predicted_classes <- ifelse(probs > optimal_cutoff, 1, 0)

table(Predicted = predicted_classes, Actual = test$stroke)

predicted_classes <- factor(predicted_classes, levels = c(0,1))
actual_classes <- factor(test$stroke, levels = c(0,1))

cm <- confusionMatrix(predicted_classes, actual_classes, positive = "1")
cm

precision <- cm$byClass["Pos Pred Value"]
recall <- cm$byClass["Sensitivity"]

F1 <- 2 * (precision * recall) / (precision + recall)
names(F1) <- "F1-score"
F1

model_cloglog <- glm(stroke ~ age + bmi + avg_glucose_level + gender + hypertension
↪  + heart_disease + ever_married + work_type + Residence_type + smoking_status,
                        data = train, family = binomial(link = "cloglog"))

probs <- predict(model_cloglog, newdata = train, type = "response")

rocobj <- roc(train$stroke, probs)

optimal_cutoff <- as.numeric(coords(rocobj, "best", best.method="youden",
↪  ret="threshold"))
optimal_cutoff

probs <- predict(model_cloglog, newdata = test, type = "response")

rocobj <- roc(test$stroke, probs)
```

```r
plot(rocobj, main="ROC kreivė (cloglog) testavimo aibei", col="blue", lwd=2,
↪  xlab="Specifiškumas", ylab="Jautrumas")


predicted_classes <- ifelse(probs > optimal_cutoff, 1, 0)
predicted_classes <- factor(predicted_classes, levels=c(0,1))
actual_classes <- factor(test$stroke, levels=c(0,1))

cm <- confusionMatrix(predicted_classes, actual_classes, positive = "1")
cm

precision <- cm$byClass["Pos Pred Value"]
recall <- cm$byClass["Sensitivity"]

F1 <- 2 * (precision * recall) / (precision + recall)
names(F1) <- "F1-score"
F1

model_probit <- glm(stroke ~ age + bmi + avg_glucose_level + gender + hypertension +
↪  heart_disease + ever_married + work_type + Residence_type + smoking_status,
                      data = train, family = binomial(link = "probit"))

probs <- predict(model_probit, newdata = test, type = "response")

rocobj <- roc(test$stroke, probs)

plot(rocobj, main="ROC kreivė (probit) testavimo aibei", col="blue", lwd=2,
↪  xlab="Specifiškumas", ylab="Jautrumas")

probs <- predict(model_probit, newdata = train, type = "response")

rocobj <- roc(train$stroke, probs)

optimal_cutoff <- as.numeric(coords(rocobj, "best", best.method="youden",
↪  ret="threshold"))
optimal_cutoff

probs <- predict(model_probit, newdata = test, type = "response")

predicted_classes <- ifelse(probs > optimal_cutoff, 1, 0)
predicted_classes <- factor(predicted_classes, levels=c(0,1))
actual_classes <- factor(test$stroke, levels=c(0,1))

cm <- confusionMatrix(predicted_classes, actual_classes, positive = "1")
```

```r
cm

precision <- cm$byClass["Pos Pred Value"]
recall <- cm$byClass["Sensitivity"]

F1 <- 2 * (precision * recall) / (precision + recall)
names(F1) <- "F1-score"
F1

model_probit <- glm(stroke ~ age + bmi + avg_glucose_level + gender + hypertension +
↪  heart_disease + ever_married + work_type + Residence_type + smoking_status,
                        data = train, family = binomial(link = "probit"))

model_probit_step <- step(model_probit, direction = "both")

probs <- predict(model_probit_step, newdata = train, type = "response")

rocobj <- roc(train$stroke, probs)

optimal_cutoff <- as.numeric(coords(rocobj, "best", best.method="youden",
↪  ret="threshold"))
optimal_cutoff

probs <- predict(model_probit_step, newdata = test, type = "response")

rocobj <- roc(test$stroke, probs)

plot(rocobj, main="ROC kreivė probit testavimo aibei", col="blue", lwd=2,
↪  xlab="Specifiškumas", ylab="Jautrumas")

predicted_classes <- ifelse(probs > optimal_cutoff, 1, 0)
predicted_classes <- factor(predicted_classes, levels=c(0,1))
actual_classes <- factor(test$stroke, levels=c(0,1))

cm <- confusionMatrix(predicted_classes, actual_classes, positive = "1")
cm

precision <- cm$byClass["Pos Pred Value"]
recall <- cm$byClass["Sensitivity"]

F1 <- 2 * (precision * recall) / (precision + recall)
names(F1) <- "F1-score"
F1
```

```r
model_probit2 <- glm(stroke ~ heart_disease * avg_glucose_level + bmi + gender + age
↪  + hypertension + ever_married + work_type + Residence_type + smoking_status,
  data = train,
  family = binomial(link = "probit")
)

probs2 <- predict(model_probit2, newdata = train, type = "response")

rocobj2 <- roc(train$stroke, probs2)

optimal_cutoff2 <- as.numeric(coords(rocobj2, "best", best.method = "youden", ret =
↪  "threshold"))
optimal_cutoff2

probs2 <- predict(model_probit2, newdata = test, type = "response")

rocobj2 <- roc(test$stroke, probs2)

plot(rocobj2,
     main = "ROC kreivė probit modeliui su sąveikomis testavimo aibei",
     col = "blue",
     lwd = 2)

predicted_classes2 <- ifelse(probs2 > optimal_cutoff2, 1, 0)

predicted_classes2 <- factor(predicted_classes2, levels = c(0,1))
actual_classes2 <- factor(test$stroke, levels = c(0,1))

cm2 <- confusionMatrix(predicted_classes2, actual_classes2, positive = "1")
cm2

precision2 <- cm2$byClass["Pos Pred Value"]
recall2 <- cm2$byClass["Sensitivity"]

F1 <- 2 * (precision2 * recall2) / (precision2 + recall2)
names(F1) <- "F1-score"
F1

conf_intervals <- confint(model_probit2)
conf_intervals

aver_marg_effects_probit <- mean(dnorm(predict(model_probit, type = "link")))
aver_marg_effects_probit * coef(model_probit)
```

Priedas B: Pilnas Python kodas

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_curve, auc, confusion_matrix, precision_score,
↪  recall_score, f1_score, accuracy_score
import statsmodels.api as sm
import statsmodels.formula.api as smf
from statsmodels.stats.outliers_influence import variance_inflation_factor

df = pd.read_csv("healthcare-dataset-stroke-data.csv")

print(df["id"].duplicated().any())

df = df.drop(columns=["id"])

df = df.dropna()

df = df[df["bmi"] != "N/A"]

df = df[df["smoking_status"] != "Unknown"]

df = df[df["gender"] != "Other"]

df = df[~df["work_type"].isin(["Never_worked", "children"])]

cols = ["gender","hypertension","heart_disease","ever_married",
        "work_type","Residence_type","smoking_status","stroke"]

for c in cols:
    print(c, df[c].unique())

print(df[["age","bmi","avg_glucose_level"]].isna().any())

print(df[["age","bmi","avg_glucose_level"]].agg(["min","max"]))

print(df["stroke"].value_counts())

df["bmi"] = pd.to_numeric(df["bmi"])
```

```
labels = {0:"Neturėjo insulto",1:"Turėjo insultą"}

df["stroke_label"] = df["stroke"].map(labels)

plt.figure()
sns.boxplot(x="stroke_label", y="bmi", data=df,
            palette=["lightgreen", "lightblue"])
plt.title("KMI pasiskirstymas pagal insultą")
plt.xlabel("Insultas")
plt.ylabel("KMI")
plt.show()

plt.figure()
sns.boxplot(x="stroke_label", y="age", data=df,
            palette=["lightgreen", "lightblue"])
plt.title("Amžiaus pasiskirstymas pagal insultą")
plt.xlabel("Insultas")
plt.ylabel("Metai")
plt.show()

plt.figure()
sns.boxplot(x="stroke_label", y="avg_glucose_level", data=df,
            palette=["lightgreen", "lightblue"])
plt.title("Vidutinio gliukozės kiekio pasiskirstymas pagal insultą")
plt.xlabel("Insultas")
plt.ylabel("Vidutinis gliukozės kiekis")
plt.show()

plt.figure()
plt.hist(df["bmi"], bins=30, color="lightgreen", edgecolor="black")
plt.title("KMI pasiskirstymas")
plt.xlabel("KMI")
plt.ylabel("Dažnis")
plt.show()

plt.figure()
plt.hist(df["age"], bins=30, color="lightblue", edgecolor="black")
plt.title("Amžiaus pasiskirstymas")
plt.xlabel("Amžius")
plt.ylabel("Dažnis")
plt.show()

plt.figure()
plt.hist(df["avg_glucose_level"], bins=30, color="lightpink", edgecolor="black")
```

```
plt.title("Vidutinis gliukozės kiekio pasiskirstymas")
plt.xlabel("Vidutinis gliukozės kiekis")
plt.ylabel("Dažnis")
plt.show()

age0 = df[df["stroke"] == 0]["age"]
age1 = df[df["stroke"] == 1]["age"]

bmi0 = df[df["stroke"] == 0]["bmi"]
bmi1 = df[df["stroke"] == 1]["bmi"]

glucose0 = df[df["stroke"] == 0]["avg_glucose_level"]
glucose1 = df[df["stroke"] == 1]["avg_glucose_level"]

plt.figure()
sns.kdeplot(age0, fill=True, alpha=0.4, color="orange", bw_adjust=1,
↪  common_norm=True)
sns.kdeplot(age1, fill=True, alpha=0.4, color="blue", bw_adjust=1, common_norm=True)
plt.title("Amžiaus tankio grafikas pagal insultą")
plt.xlabel("Amžius")
plt.ylabel("Tankis")
plt.show()

plt.figure()
sns.kdeplot(bmi0, fill=True, alpha=0.4, color="orange", bw_adjust=1,
↪  common_norm=True)
sns.kdeplot(bmi1, fill=True, alpha=0.4, color="blue", bw_adjust=1, common_norm=True)
plt.title("KMI tankio grafikas pagal insultą")
plt.xlabel("KMI")
plt.ylabel("Tankis")
plt.show()


plt.figure()
sns.kdeplot(glucose0, fill=True, alpha=0.4, color="orange", bw_adjust=1,
↪  common_norm=True)
sns.kdeplot(glucose1, fill=True, alpha=0.4, color="blue", bw_adjust=1,
↪  common_norm=True)
plt.title("Vidutinio gliukozės kiekio tankio grafikas pagal insultą")
plt.xlabel("Vidutinis gliukozės kiekis")
plt.ylabel("Tankis")
plt.show()

print(pd.crosstab(df["gender"], df["stroke"]))
```

```
print(pd.crosstab(df["ever_married"], df["stroke"]))
print(pd.crosstab(df["hypertension"], df["stroke"]))
print(pd.crosstab(df["heart_disease"], df["stroke"]))
print(pd.crosstab(df["work_type"], df["stroke"]))
print(pd.crosstab(df["Residence_type"], df["stroke"]))
print(pd.crosstab(df["smoking_status"], df["stroke"]))


df0 = df[df["stroke"] == 0]
df1 = df[df["stroke"] == 1]

np.random.seed(8)

train0 = df0.sample(frac=0.8, random_state=8)
train1 = df1.sample(frac=0.8, random_state=8)

test0 = df0.drop(train0.index)
test1 = df1.drop(train1.index)

train = pd.concat([train0, train1])
test = pd.concat([test0, test1])

print(train["stroke"].value_counts())
print(test["stroke"].value_counts())

model = smf.glm(
    formula="stroke ~ age + bmi + avg_glucose_level + gender + hypertension +
    ↪  heart_disease + ever_married + work_type + Residence_type + smoking_status",
    data=train,
    family=sm.families.Binomial()
).fit()

print(model.summary())

influence = model.get_influence()
cooksd = influence.cooks_distance[0]

outliers = cooksd[cooksd > 1]
print(outliers)

plt.figure()
plt.stem(cooksd, linefmt='navy', markerfmt=" ", basefmt=" ")
plt.title("Kuko matas")
plt.xlabel("Stebėjimai")
```

```
plt.ylabel("Cook atstumas")
plt.show()

pearson_resid = model.resid_pearson

plt.figure()
plt.scatter(range(len(pearson_resid)), pearson_resid, color="navy")
plt.title("Pearsono liekanos")
plt.xlabel("Stebėjimai")
plt.ylabel("Liekanos")
plt.axhline(3, color="red", linestyle="--")
plt.show()

outlier_idx = np.where(np.abs(pearson_resid) > 3)[0]

print(train.iloc[outlier_idx])

train_clean = train.drop(train.index[outlier_idx])

model_clean = smf.glm(
    formula="stroke ~ age + bmi + avg_glucose_level + gender + hypertension +
    ↪  heart_disease + ever_married + work_type + Residence_type + smoking_status",
    data=train_clean,
    family=sm.families.Binomial()
).fit()

print(model_clean.summary())

X = train_clean.drop(columns=["stroke"])
X = pd.get_dummies(X, drop_first=True)
X = sm.add_constant(X).astype(float)
vif_data = pd.DataFrame({"feature": X.columns,
                         "VIF": [variance_inflation_factor(X.values, i) for i in
                         ↪  range(X.shape[1])]})
print(vif_data)

probs_train = model_clean.predict(train_clean)
fpr, tpr, thresholds = roc_curve(train_clean["stroke"], probs_train)
youden_index = tpr - fpr
optimal_cutoff = thresholds[np.argmax(youden_index)]

print("Optimalus cutoff:", optimal_cutoff)

plt.figure()
```

```
plt.plot(fpr, tpr, color='darkorange', lw=2)
plt.plot([0, 1], [0, 1], color='black', lw=1, linestyle='--')
plt.title("ROC kreivė mokymo aibei")
plt.xlabel("Specifiškumas)")
plt.ylabel("Jautrumas")
plt.legend(loc="lower right")
plt.show()

probs_test = model_clean.predict(test)

predicted_classes = (probs_test > optimal_cutoff).astype(int)
cm = confusion_matrix(test["stroke"], predicted_classes)
tn, fp, fn, tp = cm.ravel()

sensitivity = recall_score(test["stroke"], predicted_classes)
specificity = tn / (tn + fp)
accuracy = accuracy_score(test["stroke"], predicted_classes)
f1 = f1_score(test["stroke"], predicted_classes)

print(f"Confusion matrix:\n{cm}\n")
print(f"Jautrumas: {sensitivity:.4f}")
print(f"Specifiškumas: {specificity:.4f}")
print(f"Tikslumas: {accuracy:.4f}")
print(f"F1 balas: {f1:.4f}")

fpr_test, tpr_test, _ = roc_curve(test["stroke"], probs_test)
plt.figure()
plt.plot(fpr_test, tpr_test, color='darkorange', lw=2)
plt.plot([0,1], [0,1], color='navy', lw=1, linestyle='--')
plt.title("ROC kreivė testavimo aibei")
plt.xlabel("Specifiškumas")
plt.ylabel("Jautrumas")
plt.show()

model_cloglog = smf.glm(
    formula="stroke ~ age + bmi + avg_glucose_level + gender + hypertension +
    ↪  heart_disease + ever_married + work_type + Residence_type + smoking_status",
    data=train_clean,
    family=sm.families.Binomial(link=sm.families.links.cloglog())).fit()
print(model_cloglog.summary())

probs_train_cloglog = model_cloglog.predict(train_clean)
fpr, tpr, thresholds = roc_curve(train_clean["stroke"], probs_train_cloglog)
youden_index = tpr - fpr
```

```
optimal_cutoff_cloglog = thresholds[np.argmax(youden_index)]

print("Optimalus cutoff (cloglog):", optimal_cutoff_cloglog)

probs_test_cloglog = model_cloglog.predict(test)
predicted_classes_cloglog = (probs_test_cloglog >
↪  optimal_cutoff_cloglog).astype(int)

cm_cloglog = confusion_matrix(test["stroke"], predicted_classes_cloglog)
tn, fp, fn, tp = cm_cloglog.ravel()
sensitivity = recall_score(test["stroke"], predicted_classes_cloglog)
specificity = tn / (tn + fp)
accuracy = accuracy_score(test["stroke"], predicted_classes_cloglog)
f1 = f1_score(test["stroke"], predicted_classes_cloglog)

print(f"\Confusion matrix:\n{cm_cloglog}")
print(f"Jautrumas: {sensitivity:.4f}")
print(f"Specifiškumas: {specificity:.4f}")
print(f"Tikslumas: {accuracy:.4f}")
print(f"F1 balas: {f1:.4f}")

fpr_test, tpr_test, _ = roc_curve(test["stroke"], probs_test_cloglog)
plt.figure()
plt.plot(fpr_test, tpr_test, color='blue', lw=2)
plt.plot([0,1], [0,1], color='navy', lw=1, linestyle='--')
plt.title("ROC kreivė (cloglog) testavimo aibei")
plt.xlabel("Specifiškumas")
plt.ylabel("Jautrumas")
plt.show()

model_probit = smf.glm(
    formula="stroke ~ age + bmi + avg_glucose_level + gender + hypertension +
    ↪  heart_disease + ever_married + work_type + Residence_type + smoking_status",
    data=train_clean,
    family=sm.families.Binomial(link=sm.families.links.probit())
).fit()
print(model_probit.summary())

probs_train_probit = model_probit.predict(train_clean)
fpr, tpr, thresholds = roc_curve(train_clean["stroke"], probs_train_probit)
youden_index = tpr - fpr
optimal_cutoff_probit = thresholds[np.argmax(youden_index)]

print("Optimalus cutoff (probit):", optimal_cutoff_probit)
```

```
probs_test_probit = model_probit.predict(test)
predicted_classes_probit = (probs_test_probit > optimal_cutoff_probit).astype(int)

cm_probit = confusion_matrix(test["stroke"], predicted_classes_probit)
tn, fp, fn, tp = cm_probit.ravel()
sensitivity = recall_score(test["stroke"], predicted_classes_probit)
specificity = tn / (tn + fp)
accuracy = accuracy_score(test["stroke"], predicted_classes_probit)
f1 = f1_score(test["stroke"], predicted_classes_probit)

print(f"\nPROBIT Confusion matrix:\n{cm_probit}")
print(f"Jautrumas: {sensitivity:.4f}")
print(f"Specifiškumas: {specificity:.4f}")
print(f"Tikslumas: {accuracy:.4f}")
print(f"F1 balas: {f1:.4f}")

fpr_test, tpr_test, _ = roc_curve(test["stroke"], probs_test_probit)
plt.figure()
plt.plot(fpr_test, tpr_test, color='blue', lw=2)
plt.plot([0,1], [0,1], color='navy', lw=1, linestyle='--')
plt.title("ROC kreivė (probit) testavimo aibei")
plt.xlabel("Specifiškumas")
plt.ylabel("Jautrumas")
plt.show()

def stepwise_selection(data, response, predictors):
    import statsmodels.formula.api as smf

    remaining = set(predictors)
    selected = []
    current_score, best_new_score = float("inf"), float("inf")

    while remaining:
        scores_with_candidates = []

        for candidate in remaining:
            formula = "{} ~ {}".format(response,
                                       " + ".join(selected + [candidate]))
            model = smf.glm(formula=formula, data=data,
                            family=sm.families.Binomial()).fit()
            score = model.aic
            scores_with_candidates.append((score, candidate))
```

```
        scores_with_candidates.sort()
        best_new_score, best_candidate = scores_with_candidates[0]

        if best_new_score < current_score:
            remaining.remove(best_candidate)
            selected.append(best_candidate)
            current_score = best_new_score
        else:
            break

    formula = "{} ~ {}".format(response, " + ".join(selected))
    model = smf.glm(formula=formula, data=data,
                    family=sm.families.Binomial()).fit()

    return model

predictors = ["age","bmi","avg_glucose_level","gender","hypertension",
              "heart_disease","ever_married","work_type",
              "Residence_type","smoking_status"]


model_probit_step = stepwise_selection(train_clean, "stroke", predictors)

print(model_probit_step.summary())

probs_train_probit_step = model_probit_step.predict(train_clean)

fpr, tpr, thresholds = roc_curve(train_clean["stroke"], probs_train_probit_step)

youden = tpr - fpr
optimal_cutoff_probit_step = thresholds[np.argmax(youden)]

print("Optimalus cutoff:", optimal_cutoff_probit_step)

probs_test_probit_step = model_probit_step.predict(test)

predicted_classes_probit_step = (probs_test_probit_step >
↪  optimal_cutoff_probit_step).astype(int)

cm_probit_step = confusion_matrix(test["stroke"], predicted_classes_probit_step)

tn, fp, fn, tp = cm_probit_step.ravel()

sensitivity = recall_score(test["stroke"], predicted_classes_probit_step)
```

```
specificity = tn / (tn + fp)
accuracy = accuracy_score(test["stroke"], predicted_classes_probit_step)
f1 = f1_score(test["stroke"], predicted_classes_probit_step)

print(f"\nConfusion matrix:\n{cm_probit_step}")
print(f"Jautrumas: {sensitivity:.4f}")
print(f"Specifiškumas: {specificity:.4f}")
print(f"Tikslumas: {accuracy:.4f}")
print(f"F1 balas: {f1:.4f}")

fpr_test, tpr_test, _ = roc_curve(test["stroke"], probs_test_probit_step)

plt.figure()
plt.plot(fpr_test, tpr_test, color='blue', lw=2)
plt.plot([0,1], [0,1], linestyle='--')
plt.title("ROC kreivė pažingsniniam probit modeliui")
plt.xlabel("Specifiškumas")
plt.ylabel("Jautrumas")
plt.show()

model_probit2 = smf.glm(
    formula="stroke ~ heart_disease * avg_glucose_level + bmi + gender + age +
    ↪  hypertension + ever_married + work_type + Residence_type + smoking_status",
    data=train_clean,
    family=sm.families.Binomial(link=sm.families.links.probit())
).fit()

print(model_probit2.summary())

probs_train_probit2 = model_probit2.predict(train_clean)

fpr, tpr, thresholds = roc_curve(train_clean["stroke"], probs_train_probit2)

youden = tpr - fpr
optimal_cutoff_probit2 = thresholds[np.argmax(youden)]

print("Optimalus cutoff:", optimal_cutoff_probit2)

probs_test_probit2 = model_probit2.predict(test)

predicted_classes_probit2 = (probs_test_probit2 >
↪  optimal_cutoff_probit2).astype(int)

cm_probit2 = confusion_matrix(test["stroke"], predicted_classes_probit2)
```

```
tn, fp, fn, tp = cm_probit2.ravel()

sensitivity = recall_score(test["stroke"], predicted_classes_probit2)
specificity = tn / (tn + fp)
accuracy = accuracy_score(test["stroke"], predicted_classes_probit2)
f1 = f1_score(test["stroke"], predicted_classes_probit2)

print(f"\nConfusion matrix:\n{cm_probit2}")
print(f"Jautrumasa: {sensitivity:.4f}")
print(f"Specifiškumas: {specificity:.4f}")
print(f"Tiksulas: {accuracy:.4f}")
print(f"F1_balas: {f1:.4f}")

fpr_test, tpr_test, _ = roc_curve(test["stroke"], probs_test_probit2)

plt.figure()
plt.plot(fpr_test, tpr_test, color='blue', lw=2)
plt.plot([0,1], [0,1], linestyle='--')
plt.title("ROC kreivė probit modeliui su sąveika")
plt.xlabel("Specifiškumas")
plt.ylabel("Jautrumas")
plt.show()

conf_intervals = model_probit.conf_int()
print(conf_intervals)

eta = model_probit.predict(train_clean, linear=True)
aver_marg_effects_probit = np.mean(sm.distributions.norm.pdf(eta))
marginal_effects = aver_marg_effects_probit * model_probit.params

print(marginal_effects)
```